HARN: Hierarchical Associative Resonance Network for Event-Driven Multi-Timeframe Forecasting

Nabeel Ahmad Saidd

Dr. A. P. J. Abdul Kalam Technical University, Lucknow, India

nabeelahmadsaidd@gmail.com

Abstract. Live financial forecasting must synchronize asynchronous bar completions across timeframes without using stale or unavailable information. We introduce HARN (Hierarchical Associative Resonance Network), an event-driven architecture that anchors each event to the finest configured timeframe and maintains persistent states and matrix-valued associative memories across temporal levels. Its timestamp-masked transition updates a level only when its completed-bar timestamp matches the current anchor, preserving inactive representations. HARN integrates causal multi-scale encoding, residual-driven gated associative writes, all-to-all cross-level resonance, and ordered bottom-up evidence aggregation to produce forecasts at each temporal level. Predictions are expressed as basis-point changes and reconstructed on the price scale for evaluation. We evaluate HARN on four assets spanning equities, foreign exchange, and gold (AAPL, EURUSD, USDCHF, and XAUUSD), using chronological 70/15/15 splits, 32-bar lookbacks, one-step horizons, and three random seeds. HARN obtains lower anchor-timeframe test mean absolute errors (MAEs) than the evaluated single-timeframe PatchTST and TimeXer baselines on all four assets. HARN's MAEs are 0.508, 0.000306, 0.000263, and 2.552, respectively; PatchTST records 2.074, 0.000759, 0.000638, and 9.563, while TimeXer records 2.882, 0.010838, 0.014144, and 15.297. These descriptive comparisons do not isolate architectural effects from differences in inputs, target formulations, or training procedures. Single-run component ablations on AAPL and EURUSD show small, non-monotonic differences. The results demonstrate HARN's feasibility for asynchronous multi-timeframe forecasting, while motivating controlled comparisons and broader ablations to establish its relative advantages and component contributions.

Keywords: financial time-series forecasting; event-driven state updates; multi-timeframe learning; associative memory; temporal state persistence

JEL Classification: C22; C45; C53; G17

1 Introduction

A forecaster operating on live financial data must account for a temporal structure that standard sequence-modeling benchmarks often abstract away: observations at different resolutions do not become available simultaneously. A five-minute bar may close while the corresponding hourly or four-hour bar is still forming. Consequently, the *event time*, when a forecast is requested, and the *representation time*, the timestamp of the latest completed bar at each timeframe, need not coincide above the finest resolution. A valid forecasting system must distinguish newly available information from representations that remain unchanged. Reading a still-forming higher-timeframe bar, or refreshing a coarse representation when no new bar has closed, changes the information set available to the model and can introduce look-ahead leakage (Bergmeir et al., 2018; Kaufman et al., 2012; Tashman, 2000).

This issue is especially important in financial time series, where volatility clustering, changing conditional dependence, and heavy-tailed returns contribute to a non-stationary statistical environment (Bollerslev, 1986; Cont, 2001; Engle, 1982). Regime-switching and state-space models provide established representations of evolving market dynamics (Hamilton, 1994; Tsay, 2010), whereas modern neural methods learn temporal representations directly from historical observations. Regardless of the modeling paradigm, a live multi-timeframe system must first specify which information is available at each event and when each temporal representation may change.

Most neural forecasting architectures map fixed windows to future values. Recurrent models maintain sequential state (Hochreiter and Schmidhuber, 1997; Voelker et al., 2019), and state-space models provide alternative mechanisms for persistent temporal representation (Gu and Dao, 2024; Gu, Goel, et al., 2022; Smith et al., 2023). These approaches generally advance their state whenever the input sequence advances; they do not, by themselves, represent a hierarchy in which an hourly representation remains unchanged across several intervening five-minute events. Transformers provide flexible interactions across historical windows (Vaswani et al., 2017), but attention over resampled or regularly indexed contexts does not specify when asynchronous representations should be refreshed. Resampling, concatenation, or forward filling places coarse information on a common grid, yet does not distinguish a newly completed bar from a carried-forward value. Training an independent model for each timeframe avoids this ambiguity but forfeits an integrated mechanism for cross-scale interaction.

These observations motivate an event-driven formulation in which each timeframe maintains its own representation and updates it only when new information at that resolution becomes available. Such a formulation requires three ingredients: causal observation alignment, persistent state between updates, and an explicit rule for communication across temporal levels. Prior work provides relevant building blocks. Financial econometric models describe evolving conditional dependence and volatility (Hamilton, 1994; Tsay, 2010); neural forecasting architectures exploit long temporal contexts through attention, patching, decomposition, and frequency-aware representations (Nie et al., 2023; Wang et al., 2024; Wu, Hu, et al., 2023; Wu, Xu, et al., 2021; H. Zhou et al., 2021; T. Zhou et al., 2022); and associative-memory approaches provide learned key–value storage and retrieval (J. Ba et al., 2016; Hopfield, 1982; Ramsauer et al., 2021; Schlag et al., 2021). These components motivate the design

developed here; we do not claim an exhaustive distinction from all prior multi-timeframe or event-driven systems.

We introduce HARN, a Hierarchical Associative Resonance Network for event-driven multi-timeframe forecasting. HARN treats the fastest configured timeframe as an anchor event stream and maintains a persistent state for every configured temporal level. Each level has a causal multi-scale encoder, a gated associative memory, and a forecasting head. At an anchor event, the model receives the most recent completed window available at each level. A level's state and memory are updated only when its completed-bar indicator fires; otherwise, its previous representation is carried forward. States available at the same event interact through cross-level resonance, while a directional bottom-up readout passes information from lower to higher temporal levels. Forecasting is performed in basis-point change space, and forecasts are reconstructed to the original price scale for evaluation. HARN is thus an event-driven state-transition system, of which the neural components described here are one parameterization.

The central contribution is an explicit formulation and implementation of persistent, asynchronously updated multi-timeframe representations, not a claim of universal predictive superiority. We evaluate HARN on four assets (AAPL, EURUSD, USDCHF, and XAUUSD) using the accompanying preprocessing, training, and evaluation pipeline. The analysis reports reconstructed-price forecasts, comparisons with single-timeframe PatchTST and TimeXer baselines, and component ablations. Because the baselines are trained only on the anchor timeframe, their comparison with HARN is descriptive rather than information-matched; differences in input information, target formulation, and training procedure all affect its interpretation. A code-level audit further examines consistency with the event-level causal protocol. This audit is a static code review, not an empirical leakage test.

The contributions of this paper are as follows:

1. We formalize an event-driven multi-timeframe forecasting protocol based on completed-bar alignment, anchor events, persistent per-level state, and asynchronous update indicators, including its batched training formulation.
2. We develop HARN as a neural parameterization of this protocol, combining causal multi-scale encoding, gated associative memory, cross-level resonance, and directional bottom-up evidence aggregation.
3. We report the forecasting and ablation results with explicit experimental scope and provenance, distinguishing descriptive baseline comparisons from component-level observations.
4. We provide a code-level causal-consistency audit and document the available preprocessing, configuration, checkpoint, and per-seed result artifacts to support reproducibility.

The remainder of the paper is organized as follows. Section 2 reviews related work. Sections 3 to 5 formalize the forecasting protocol, HARN architecture, and streaming and training procedures. Sections 6 to 8 describe the experimental setup, forecasting results, and ablation analysis. Section 9 reports the implementation audit, while Sections 10 and 11 discuss limitations and identify directions for further validation.

2 Related Work

Financial forecasting begins in a setting where the statistical properties of the observed process are neither fixed nor necessarily stable. Prices, returns, volatility, and trading activity exhibit different forms of dependence, and the efficient-markets literature sets a demanding benchmark: persistent predictability must be demonstrated empirically rather than assumed (Fama, 1970). Financial returns exhibit heavy tails, volatility clustering, and changing dependence structures (Cont, 2001). ARCH and GARCH models formalize time-varying conditional variance (Bollerslev, 1986; Engle, 1982), while state-space and regime-switching formulations allow the underlying data-generating process to evolve (Hamilton, 1994; Tsay, 2010). Under these conditions, evaluation protocol is as important as model capacity. A forecasting system can exploit scale artifacts, normalization statistics, temporal dependence, or a favorable subperiod without learning a relationship that remains available at deployment. Random or otherwise inappropriate validation schemes can therefore misrepresent forecasting performance in non-stationary settings (Bergmeir et al., 2018).

This concern extends beyond the train–validation–test split. Temporal leakage may arise through feature construction, normalization, resampling, target alignment, hyperparameter selection, or inadvertent reuse of future information (Kaufman et al., 2012). Forecast evaluation must respect both dependence among forecast errors and the information set available at each forecast origin (Bergmeir et al., 2018; Tashman, 2000). No single scale-free error measure is uniformly appropriate across forecasting problems (Hyndman and Koehler, 2006). Comparative evaluation may therefore require tests based on loss differentials across dependent forecast origins, such as the Diebold–Mariano framework (Diebold and Mariano, 1995), or conditional predictive-ability tests when relative performance depends on the information set (Giacomini and White, 2006). For probabilistic forecasts, proper scoring rules provide the corresponding basis for evaluating predictive distributions (Gneiting and Raftery, 2007). This work adopts the implementation-level implications of that literature: chronological separation, training-only normalization, completed-bar alignment, target isolation, causal feature construction, explicit state-reset boundaries, and validation-only checkpoint selection. The code-level audit in Section 9 examines these conditions. The forecasting target is a basis-point change rather than an absolute price level, following the broader practice of modeling returns and related transformations (Cont, 2001; Tsay, 2010). This is a design rationale, not an experimentally established advantage, because the experiments do not isolate target representation.

Within this setting, neural sequence models have substantially expanded the class of temporal dependencies that can be represented. DeepAR learns shared recurrent probabilistic models across related series (Salinas et al., 2020); N-BEATS constructs forecasts through backcast and forecast transformations (Oreshkin et al., 2020); and Temporal Fusion Transformers combine recurrent processing, attention, and variable selection for multi-horizon forecasting (Lim et al., 2021). The Transformer introduced global self-attention as an alternative to recurrence (Vaswani et al., 2017), and subsequent time-series architectures explored sparse attention, decomposition, frequency-domain representations, patch-based tokenization, temporal reshaping, and cross-attention. Informer reduces attention cost through sparse attention and distillation (H. Zhou et al., 2021); Autoformer and FEDformer

introduce decomposition-based approaches with autocorrelation or frequency-enhanced attention (Wu, Xu, et al., 2021; T. Zhou et al., 2022); PatchTST represents time-series segments as patches (Nie et al., 2023); TimesNet maps temporal variation into a two-dimensional representation (Wu, Hu, et al., 2023); and TimeXer uses cross-attention to incorporate exogenous variables (Wang et al., 2024). Controlled multi-asset benchmarking further illustrates the value of comparing such architectures under a common forecasting setup (Saidd, 2026a).

These architectures are strong alternatives for temporal representation learning, but their usual forecasting protocol assumes a preconstructed sequence or covariate panel whose indices define temporal progression. In a multi-timeframe financial stream, representations become informative at different event times: a coarse bar may remain unchanged across several fine-scale events, whereas a newly completed fine-scale bar may require an immediate update. Treating these observations as a regular sequence therefore requires an explicit rule for when each representation is refreshed and how an inactive representation is carried forward. HARN makes this rule part of the model specification rather than an external preprocessing assumption. Each timeframe has a persistent state and an update indicator; when the indicator is inactive, the corresponding state and associative memory are preserved rather than overwritten (Eq. (2)). Cross-level interaction then proceeds through resonance (Eq. (22)) and a defined lower-to-higher evidence pathway (Eq. (23)). This distinction concerns forecasting protocol and state-transition semantics, not a rejection of attention-based modeling: HARN itself uses attention-like interactions across levels and in its evidence readout.

Persistent state is also central to recurrent and state-space approaches. LSTM introduced gated memory mechanisms that substantially improved the representation of long-range dependencies (Hochreiter and Schmidhuber, 1997). Legendre Memory Units encode long histories through structured continuous-time dynamics (Voelker et al., 2019), while structured state-space models provide explicit state-transition formulations with efficient long-sequence computation (Gu and Dao, 2024; Gu, Goel, et al., 2022; Smith et al., 2023). Selective state-space models, including Mamba, further condition state transitions on the input itself (Gu and Dao, 2024). HARN is related to this family through its compact persistent states, but it makes the update event an explicit component of the transition rule. Conventional recurrent formulations ordinarily advance a state at every sequence index; HARN instead applies a per-timeframe mask, allowing one level to remain unchanged while other levels receive new observations (Eq. (2)). Its state is therefore a coupled collection of level-specific recurrences operating under an event-conditioned schedule. Recurrent and state-space models are relevant architectural alternatives, but they are not tested controls for the event-conditioned transition mechanism studied here.

A second relevant line of work concerns associative memory. Hopfield networks established content-addressable memory through an energy-based attractor formulation (Hopfield, 1982). Fast-weight methods subsequently introduced temporary key–value structures that can be updated to encode recent context (J. Ba et al., 2016), and linear Transformers admit an interpretation in terms of fast-weight programming and associative updates (Schlag et al., 2021). Modern Hopfield networks connect associative retrieval with attention-like mechanisms and substantially extend the representational capacity of the original formulation (Ramsauer et al., 2021). HARN uses a narrower mechanism

designed for its event-driven setting. Its encoder output and cross-scale context yield a normalized key, a value, and a normalized query; the difference between the current value and the retrieved memory content forms a retrieval residual, referred to as "surprise" in Section 4.2, which controls a bounded outer-product write (Eq. (17)). The mechanism thus separates retrieval from writing rather than treating memory solely as another recurrent hidden-state transformation. The A1 ablation replaces this associative cell with a gated recurrent-style update; because that comparison is not parameter-count matched, it does not isolate memory capacity as a single causal factor (Section 8).

The temporal hierarchy considered by HARN is also related to hierarchical forecasting, although the objectives differ. Hierarchical forecasting concerns forecasts defined at multiple aggregation levels and, in many formulations, their reconciliation into a coherent collection; optimal-combination methods construct forecasts that respect aggregation relationships (Hyndman, Ahmed, et al., 2011). Neural architectures such as N-BEATS and Temporal Fusion Transformers learn multi-scale or multi-horizon representations (Lim et al., 2021; Oreshkin et al., 2020), and decomposition-based Transformer models similarly separate temporal variation at different scales (Wu, Xu, et al., 2021; T. Zhou et al., 2022). HARN uses the term "hierarchical" in a related but distinct sense. Its hierarchy consists of timeframes whose bars complete at different rates, and it determines when a persistent representation is eligible for a state transition rather than imposing an output-side aggregation constraint. A coarse representation is not merely a lower-frequency forecasting head: it retains its own state while receiving contemporaneous cross-level context and lower-level evidence. Conversely, repeating the most recent coarse value does not record whether a new coarse bar has actually completed, so HARN treats completion status as explicit information through the update indicators. The reported experiments do not compare this mechanism with reconciled hierarchical forecasting methods or with explicitly matched multi-scale neural baselines.

Irregular and event-driven sequence models offer another point of comparison. Latent ODEs model hidden trajectories at arbitrary observation times by integrating a learned vector field (Rubanova et al., 2019), and Neural Controlled Differential Equations treat an observed path as a control signal driving continuous hidden dynamics (Kidger et al., 2020). These approaches handle observations that are not regularly spaced in a principled way. The multi-timeframe financial setting considered here is more structured: each timeframe has a known bar-completion rule, observations have ordered timestamps, each input window contains completed bars, and the update status of a level follows from timestamp equality. HARN consequently uses discrete event ordering and state retention rather than inferring a continuous hidden trajectory between observations. This makes the causal contract directly inspectable, but it also makes the implementation dependent on the specified timestamp and bar-construction conventions, which are examined in Section 9.

Taken together, these literatures supply the components from which HARN is constructed. Financial econometrics motivates careful treatment of non-stationarity and information availability (Cont, 2001; Hamilton, 1994); recurrent and state-space models provide mechanisms for compact persistent state (Gu and Dao, 2024; Hochreiter and Schmidhuber, 1997); Transformers provide flexible cross-temporal interaction (Nie et al., 2023; Vaswani et al., 2017); associative-memory methods provide content-addressable retrieval and update (Hopfield, 1982; Schlag et al., 2021); hierarchical forecasting addresses

relationships across temporal aggregation levels (Hyndman, Ahmed, et al., 2011); irregular-time models offer alternatives for asynchronous observations (Kidger et al., 2020; Rubanova et al., 2019); and leakage and forecast-evaluation research establishes the need to align computation with the information set available at each forecast origin (Bergmeir et al., 2018; Diebold and Mariano, 1995; Kaufman et al., 2012). The more specific gap addressed by HARN lies in how these ideas are combined for a stream of completed financial bars observed at multiple resolutions.

To our knowledge, the literature reviewed here does not provide a single established protocol that simultaneously specifies per-timeframe completion events, persistence of inactive higher-level states, within-event cross-level communication, and compact associative state updates under an explicit causal ordering. We frame this as a gap in protocol and state-transition specification, not as a claim that prior methods lack any individual component. HARN addresses the combination through an event-conditioned state transition, a bounded residual-driven associative write (Eq. (17)), cross-level resonance around selective updates (Eq. (22)), and a lower-to-higher evidence pathway with an explicit history order (Eq. (23)). The result is a testable specification for event-driven multi-timeframe state rather than a collection of isolated architectural claims. Accordingly, the empirical evidence is interpreted within the scope of this specification and its single implementation, not as evidence of superiority over the broader forecasting literature (Sections 6, 7 and 10).

3 Problem Formulation: Event-Driven Multi-Timeframe Forecasting

This section formalizes the protocol that Section 2 identified as under-specified. It fixes the notation used throughout and states what a causally valid multi-timeframe forecast may and may not depend on; this is the standard against which Section 9 checks the implementation.

Let the levels be ordered from finest to coarsest as $T_1, \dots, T_L$. A completed bar at level k is $x_t^k = (o_t^k, h_t^k, \ell_t^k, c_t^k, v_t^k) \in \mathbb{R}^5$, the usual open–high–low–close–volume representation of a fixed time interval. *Event time* τ is the sequence of anchor timestamps at which the system is queried for a forecast, and *representation time* is the timestamp of the most recently completed bar available at each level. The two clocks coincide at the finest level by construction, since the anchor stream is defined by that level's bar completions, but they need not coincide at coarser levels. We write r_τ^k, M_τ^k for the persistent state immediately before processing event τ, and $r_{\tau+1}^k, M_{\tau+1}^k$ for the state stored immediately after that event and used by the forecast emitted at τ. The subscript $\tau + 1$ thus denotes the post-event state; it does not imply that the forecast waits for the next event.

Update indicator. Events are indexed by the finest level. At anchor time τ, preprocessing selects the latest level-k bar whose timestamp is no later than τ and sets

$$u_\tau^k = \mathbf{1}\{\text{selected timestamp equals } \tau\}. \tag{1}$$

The no-later-than rule prevents a future bar from entering an input window, while the equality test determines separately whether the persistent level state may be overwritten. Thus, $u_\tau^k = 0$ means

that the selected higher-level window remains available as context, but no new completed bar closed exactly at τ; the level contributes its most recent valid representation and its persistent state is preserved. Exact timestamp equality is an implementation assumption, not a claim that every market feed exposes perfectly aligned timestamps; Section 9 revisits this assumption.

Persistent state and masked transition. For each level, HARN maintains a state $r_\tau^k \in \mathbb{R}^{d_s}$ and a matrix memory $M_\tau^k \in \mathbb{R}^{d_s \times d_m}$. The state is a compact vector summary analogous to a recurrent hidden state, whereas the memory is a matrix-valued associative store that supports content-based retrieval, as in fast-weight and Hopfield-style memories (J. Ba et al., 2016; Hopfield, 1982; Ramsauer et al., 2021). Let $\tilde{r}_{\tau+1}^k$ and $\tilde{M}_{\tau+1}^k$ denote candidate updates computed from the current payload: the state and memory that the level would adopt if it were active at this event. For a single stream and $k > 1$, the persistent transition is

$$\begin{aligned} r_{\tau+1}^k &= (1 - u_\tau^k)\, r_\tau^k + u_\tau^k\, \tilde{r}_{\tau+1}^k, \\ M_{\tau+1}^k &= (1 - u_\tau^k)\, M_\tau^k + u_\tau^k\, \tilde{M}_{\tau+1}^k, \end{aligned} \tag{2}$$

where $u_\tau^k \in \{0, 1\}$ is a scalar that multiplies the whole state or memory. The base level, whose bar closes at every anchor event, has $u_\tau^1 \equiv 1$ and is updated without masking. Because the indicator is binary rather than learned, this convex combination expresses "no new information at this level, so do not change its state" as an exact identity transition.

Batched form used in training. Training and evaluation process B contiguous streams in parallel (Section 5). Let $b \in \{1, \dots, B\}$ index streams, and let $u_{b,\tau}^k$ be the indicator in Eq. (1) for stream b at its current event. We use τ for the step index and suppress each stream's separate timestamp. The indicators at one step form the mask vector

$$m_\tau^k = \left(u_{1,\tau}^k, \dots, u_{B,\tau}^k\right)^\top \in \{0, 1\}^B. \tag{3}$$

With stacked states $R_\tau^k \in \mathbb{R}^{B \times d_s}$ and memories $\mathcal{M}_\tau^k \in \mathbb{R}^{B \times d_s \times d_m}$, the implemented update is

$$\begin{aligned} R_{\tau+1}^k &= (1 - m_\tau^k) \odot_B R_\tau^k + m_\tau^k \odot_B \tilde{R}_{\tau+1}^k, \\ \mathcal{M}_{\tau+1}^k &= (1 - m_\tau^k) \odot_B \mathcal{M}_\tau^k + m_\tau^k \odot_B \tilde{\mathcal{M}}_{\tau+1}^k, \end{aligned} \tag{4}$$

where $\odot_B$ multiplies the b-th slice of a tensor by the b-th entry of the mask, broadcasting over the trailing dimensions. Slice b of Eq. (4) is exactly Eq. (2) for stream b, so the scalar definition and the batched implementation coincide. The candidates $\tilde{R}$ and $\tilde{\mathcal{M}}$ are computed for the entire batch regardless of the mask; masking then selects between the two already-computed branches.

Causal requirement. Let $\mathcal{I}_\tau = \{x_s^j : s \le \tau,\ 1 \le j \le L\}$ denote the completed bars at all levels with timestamps no later than τ. A causally valid forecaster must satisfy

$$\hat{y}_{\tau+1}^k = f_k\left(\mathcal{I}_\tau;\, r_0^{1:L}, M_0^{1:L}\right), \tag{5}$$

where $(r_0^{1:L}, M_0^{1:L})$ is the reset initial state and the dependence on $\mathcal{I}_\tau$ is mediated by the persistent states obtained by applying the transition to completed bars in order. The requirement is event-level: no bar completed after τ, and no state derived from such a bar, may influence $\hat{y}^k_{\tau+1}$. It does not require the level-k update at event τ to be independent of computations at other levels during that event. HARN's bottom-up evidence path allows a higher-level update at τ to read the lower-level state updated earlier in the same event, and the forecasting head consumes the post-update states $r^k_{\tau+1}$ (Section 4). HARN therefore satisfies Eq. (5) at event granularity, but it is not past-event-only within an event. This qualification recurs as entry 9 of the audit (Table 6). HARN does enforce a stronger local property for its convolutions and shifted features (Section 4).

Implementation notes and data handling. The implementation computes candidate encodings for every supplied level on each forward call, including inactive levels; masking governs persistent overwrites, not computation skipping. "Inactive" therefore means that a level's persistent state is protected, not that its window is untouched. Within an event, lower-level updates finish before their current reads are appended to the higher-level history buffer. Data are cleaned by parsing timestamps, removing invalid rows, retaining the last duplicate, and sorting in ascending order. Splits are chronological 70%/15%/15% (Bergmeir et al., 2018; Tashman, 2000); feature and target statistics are computed on training samples only and reused for validation and test (Kaufman et al., 2012). The resulting formulation is an event-driven state-transition problem, not a regular-grid forecasting problem with additional covariates.

Stationary target. Let P^k_τ be the latest observed close and $P^k_{\tau+1}$ the next completed close at level k. The supervised target is the basis-point change

$$y^k_{\tau+1} = 10{,}000 \left(\frac{P^k_{\tau+1}}{P^k_\tau} - 1 \right) \text{ bps}, \tag{6}$$

and the prediction is reconstructed as $\hat{P}^k_{\tau+1} = P^k_\tau (1 + \hat{y}^k_{\tau+1}/10{,}000)$. Modeling a relative change rather than a level is standard practice in financial time-series analysis, since levels differ widely across assets and are highly persistent, whereas relative changes are closer to stationary (Cont, 2001; Tsay, 2010). Recent objectives also combine volatility normalization with direction-aware terms (Saidd, 2026c); HARN instead uses unnormalized per-level BPS mean-squared error, as specified in Eq. (25). The BPS target is dimensionless with respect to price level and is expressed in the units conventionally used for small relative movements. Two consequences matter for interpreting the reported results. First, the reconstruction error is

$$\hat{P}^k_{\tau+1} - P^k_{\tau+1} = \frac{P^k_\tau \, (\hat{y}^k_{\tau+1} - y^k_{\tau+1})}{10{,}000}, \tag{7}$$

so HARN's reported price errors are price-scaled BPS errors anchored at the last observed close, and a zero-BPS forecast reproduces the last-value (persistence) forecast. Second, all reported evaluation metrics are computed on reconstructed absolute price, which reintroduces asset-scale dependence; the BPS target is used for training and reconstruction and is not itself reported as an evaluation metric.

4 The HARN Architecture

Figure 1 summarizes the computation. At each event, the path is: completed OHLCV windows → per-level causal encoders → prior-state resonance → gated memory candidates → ordered bottom-up evidence for higher levels → posterior resonance → per-level forecasting heads. HARN instantiates separate encoders, memories, histories, and heads for every configured level. The hierarchy is therefore both representational and operational: level identity determines what a state represents and when it may change. This section presents the components in execution order; Section 5 then assembles them into the complete event protocol. The descriptions below concern implemented computation and are code observations, not experimental findings. In the notation, t indexes positions within an encoder window, whereas the memory, state, and head equations use the event index τ from Section 3; x_t denotes the memory input vector in those equations.

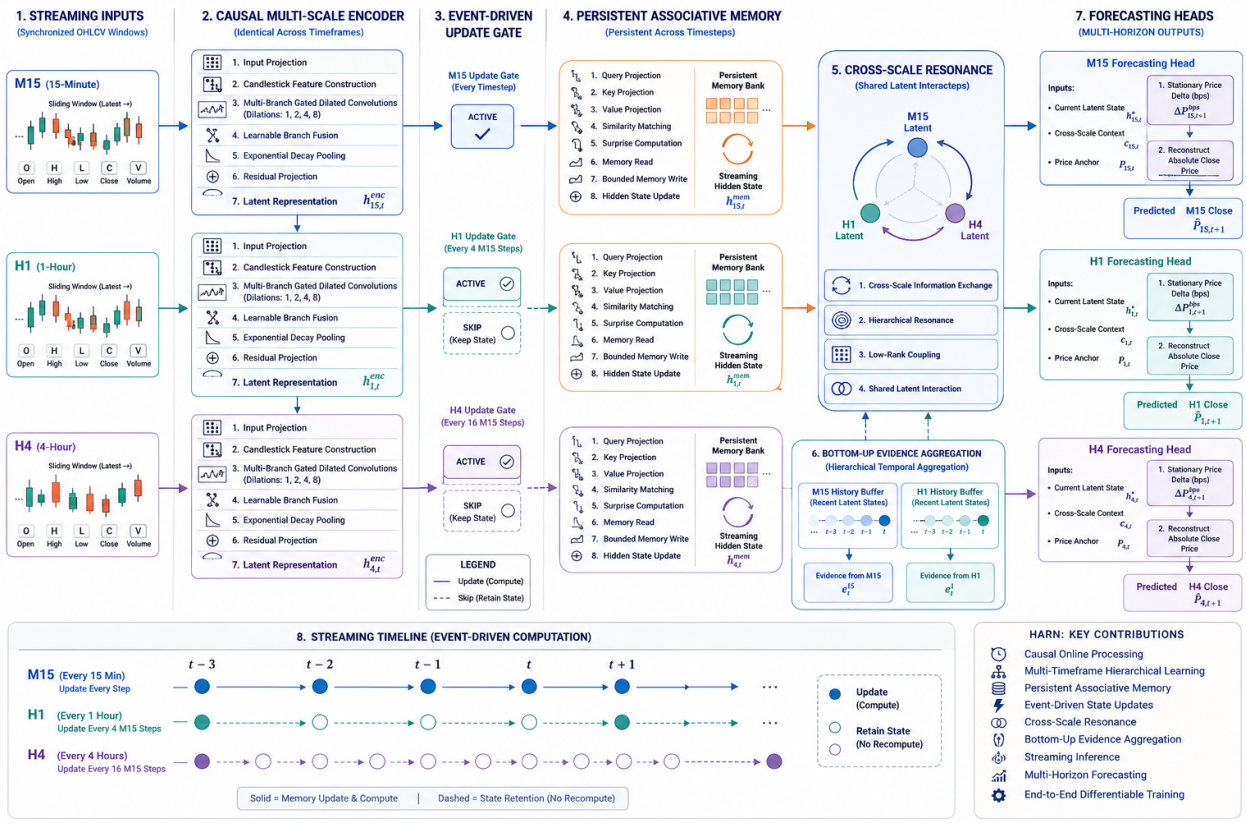


Figure 1. HARN's event-driven computational path at one anchor event: per-level causal encoding, prior-state resonance, gated memory candidates, ordered bottom-up evidence, posterior resonance, and per-level forecasting. This conceptual schematic is not an empirical result.

4.1 Causal multi-scale encoder

The encoder summarizes a completed OHLCV window as a fixed-size representation at multiple temporal scales. It uses only information within that window and never mixes in a later position

(Figure 2). For $X^k \in \mathbb{R}^{B\times T\times 5}$, it projects the raw OHLCV features together with six derived features that expose short-horizon price structure to the downstream convolutions:

$$d_t^k = (c_t - c_{t-1},\ c_t - o_t,\ h_t - \ell_t,\ h_t - \max(o_t, c_t),\ \min(o_t, c_t) - \ell_t,\ v_t), \tag{8}$$

$$z_t^k = W_{\text{raw}} x_t^k + W_{\text{derived}} d_t^k, \tag{9}$$

where the previous close is shifted by one position and the first previous close is copied from the first close, avoiding an artificial pre-window observation. The implementation receives normalized OHLCV windows and computes these quantities from the normalized representation. The six derived channels are close change, signed candle body, full candle range, upper wick, lower wick, and the already normalized volume channel; volume is therefore reused rather than transformed a second time. These are candle-geometry and volume features, not a conventional technical-indicator panel, a distinction examined in related indicator-ablation work (Saidd, 2026b). HARN does not isolate the contribution of individual derived channels.

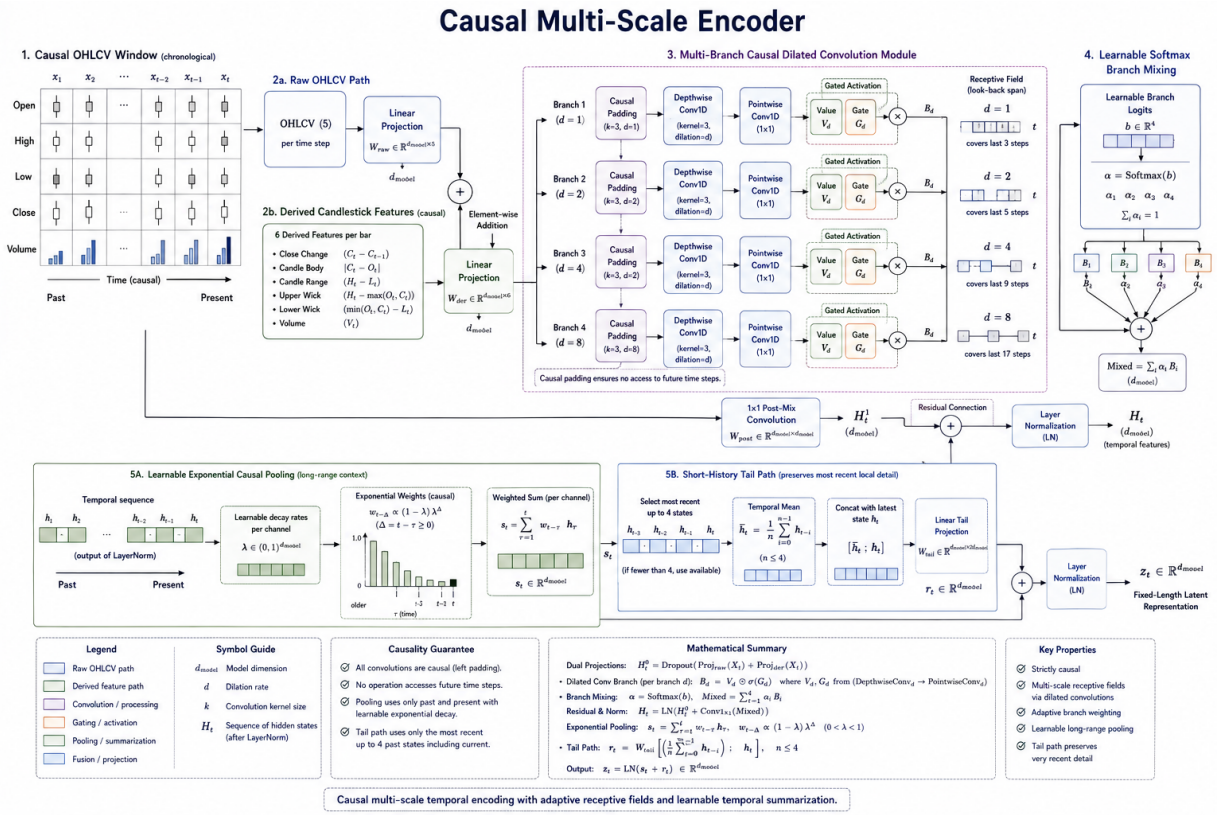


Figure 2. Causal multi-scale encoder with left-padded dilated branches, a learned branch mixture, and decayed pooling. This conceptual schematic is not an empirical result.

Four depthwise-separable gated convolution branches use kernel size three and dilations $D = \{1, 2, 4, 8\}$. Left-only padding ensures that no output at position t depends on a later position. This follows the causal dilated design used for autoregressive modeling of long sequences, in which exponentially increasing dilation expands the receptive field while the depth grows logarithmically

with sequence length (Oord et al., 2016); depthwise separation further reduces parameters relative to a full convolution (Chollet, 2017):

$$C_d = V_d \odot \sigma(G_d), \qquad C = \sum_{d \in D} \mathrm{softmax}(\alpha)_d C_d. \tag{10}$$

The gate $\sigma(G_d)$ allows each branch to modulate its contributing channels and positions. The learned softmax mixture over dilations weights temporal scales rather than averaging them uniformly. The mixture then passes through a pointwise convolution, a residual connection, and LayerNorm (J. L. Ba et al., 2016). A learnable per-channel decay $\lambda_c = \sigma(\phi_c)$ pools the sequence with greater weight on recent positions while retaining a causal contribution from the full window:

$$p_c = \sum_{t=1}^{T} (1 - \lambda_c) \lambda_c^{T-t} C_{t,c}. \tag{11}$$

The encoder output combines this decayed pool with a tail projection of the last position and a short trailing mean, so that the most recent bar and the immediate local trend are represented directly,

$$e^k = \mathrm{LN}\left(p^k + W_{\text{tail}}\left[C_T^k; \mathrm{mean}(C^k_{T-\min(4,T)+1:T})\right]\right) \in \mathbb{R}^{72} \tag{12}$$

in the default configuration. This local representation is the only window-derived signal that enters the memory cell; persistence across events enters through the memory cell described next.

4.2 Gated associative memory with residual-driven writes

The encoder summarizes the current window; the memory cell maintains a persistent, content-addressable representation across events (Figure 3). Its input concatenates the encoder output with prior resonance context and, for levels above the base, bottom-up evidence. The resulting input dimension is 144 at the base level and 216 at higher levels. For normalized input x_t, state r_t, and memory M_t, the cell computes a key, value, and query in the manner of a standard attention or fast-weight mechanism (Schlag et al., 2021):

$$k_t = \mathrm{normalize}(W_k x_t), \qquad v_t = \tanh(W_v x_t), \qquad q_t = \mathrm{normalize}(W_q x_t), \tag{13}$$

$$\bar{v}_t = M_t k_t, \qquad s_t = v_t - \bar{v}_t. \tag{14}$$

The residual s_t measures the discrepancy between the value implied by the current input and the value retrieved from memory for the corresponding key. We use "surprise" only as a label: this is an internal retrieval residual, not the downstream forecast error. No reported experiment tests whether it tracks prediction error, novelty, or market events. With $g_t = [x_t; r_t; \bar{v}_t]$, learned gates control the write and blend rates:

$$\eta_t = \sigma(W_\eta g_t), \qquad \rho_t = \sigma(W_\rho g_t), \qquad \eta_t, \rho_t \in \mathbb{R}^{B \times d_s}. \tag{15}$$

The residual enters the *content* of the write, not the gate: s_t is not an explicit input to η_t or ρ_t. The write is a rank-one, tanh-bounded outer product of the residual and the key, scaled by the square root of the memory-key dimension as in scaled dot-product attention (Vaswani et al., 2017):

$$\Omega_t = \tanh\left(\frac{s_t k_t^\top}{\sqrt{d_m}}\right), \tag{16}$$

$$M_{t+1} = (1 - \eta_t) \odot_B M_t + \eta_t \odot_B \Omega_t. \tag{17}$$

Here $M_t, \Omega_t \in \mathbb{R}^{B \times d_s \times d_m}$, while each gate has shape (B, d_s) and is unsqueezed to $(B, d_s, 1)$ before multiplication. In this equation, $\odot_B$ therefore denotes row-wise broadcasting over the d_m memory-key columns; it is not a single scalar gate for the whole matrix. The updated memory is then queried with the query vector rather than the key just written, so that retrieval reflects the memory's content as a whole, and the state update combines a memory-derived and a direct pathway,

$$\mu_t = M_{t+1} q_t, \tag{18}$$

$$r_{t+1} = \mathrm{LN}\left(r_t + \rho_t \odot W_r \mu_t + (1 - \rho_t) \odot \tanh(W_c g_t)\right), \tag{19}$$

followed by dropout (Srivastava et al., 2014). The default memory shape is $(B, 72, 18)$. Unlike a standard recurrent cell, which compresses temporal information into a single vector transition, this cell maintains a matrix of key–value associations and exposes the retrieval residual explicitly, in a manner structurally related to Hopfield-style retrieval (Hopfield, 1982; Ramsauer et al., 2021). Relative to generic fast-weight formulations (J. Ba et al., 2016; Schlag et al., 2021), the write is bounded by tanh, scaled by the memory-key dimension, and modulated by a state-sized gate. These choices are *intended* to keep the persistent memory numerically stable over long streams, but no stability analysis or capacity claim is made or tested. The A1 ablation (Section 8) replaces this cell with a simpler gated recurrent update.

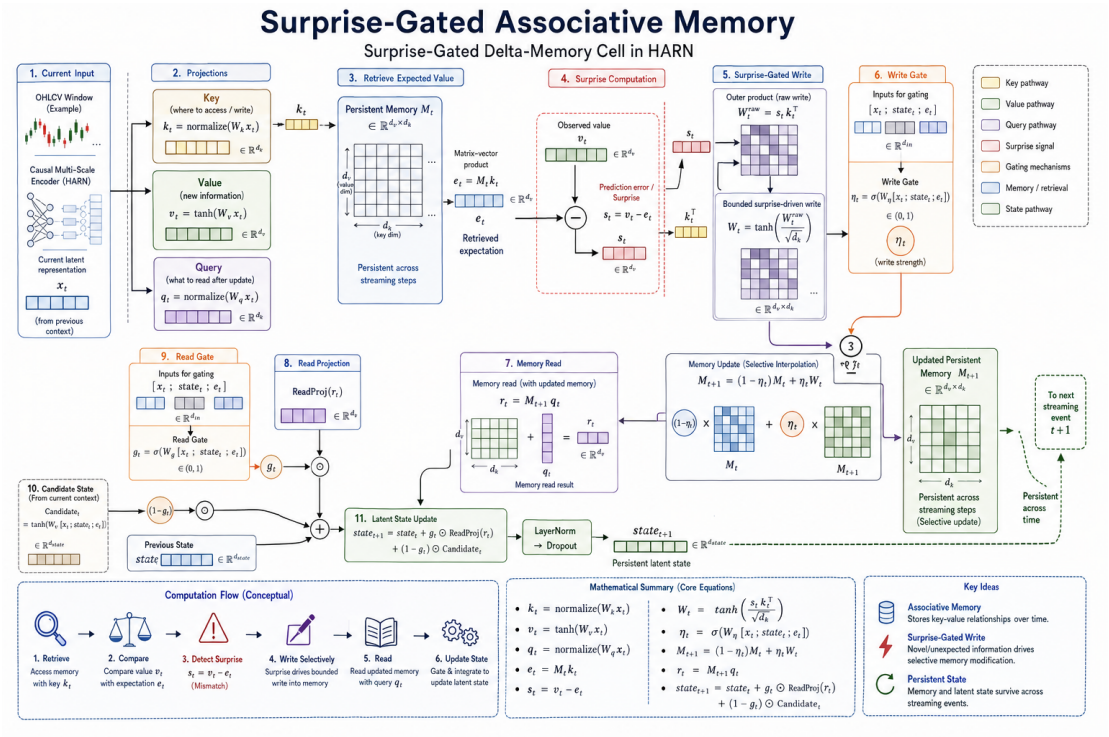


Figure 3. Gated associative memory cell showing key, value, and query projections, the retrieval residual s_t, and the bounded gated write in Eq. (17). This conceptual schematic is not an empirical result.

4.3 Cross-scale communication and forecasting head

HARN communicates across levels through two paths, resonance and evidence, which differ in topology and timing (Figure 4). For resonance, states are stacked as $R \in \mathbb{R}^{B\times L\times d_s}$, and shared projections of rank $d_r = 36$ produce an all-to-all attention update along the level axis. The operation follows multi-head self-attention (Vaswani et al., 2017), but applies it across levels rather than across time and includes a learned pairwise level-strength bias S:

$$Q = W_q R, \qquad K = W_k R, \qquad V = W_v R, \tag{20}$$

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_r}} + S\right), \tag{21}$$

$$R' = \mathrm{LN}\left(R + \mathrm{Dropout}(W_o AV)\right). \tag{22}$$

Resonance is applied twice at each event. The first application forms prior context before the memory update, so each level's write can use the other levels' states at the start of the event. The second application operates on the updated states before forecasting. Resonance is all-to-all and contemporaneous: every configured level can attend to every other level at the same event, without a directional constraint.

Bottom-up evidence is instead adjacent-level and directional: a coarser level's update uses information from the immediately finer level. For each adjacent pair, a length-four history buffer stores recent lower-level reads. A learned query attends to projected keys and values with a learnable recency

bias. The readout combines the attention-weighted summary with explicit contributions from the most recent read and the buffer mean:

$$E_i = \mathrm{LN}\left(\mathrm{Attention}(H_i) + 0.35\, H_{i,\mathrm{latest}} + 0.15\, \mathrm{mean}(H_i)\right). \tag{23}$$

The coefficients 0.35 and 0.15 are hand-set implementation constants, not learned parameters. The available materials contain no derivation, tuning record, or sensitivity analysis for them, and it is not documented whether they were tuned or inherited from earlier code. We therefore report them as manually chosen rather than empirically validated; the A3 variant zeros the entire readout and does not probe them individually.

The current lower-level read is appended to H_i before the corresponding higher-level evidence is computed, establishing an ordered same-event path. This is the qualification stated in Section 3: the path is causal with respect to future events but not restricted to past event states, because it appends the lower level's already completed update. Neither path is computation-free for inactive levels: their encoders and history reads are evaluated, while only persistent state and memory overwrites are masked (Eq. (2)).

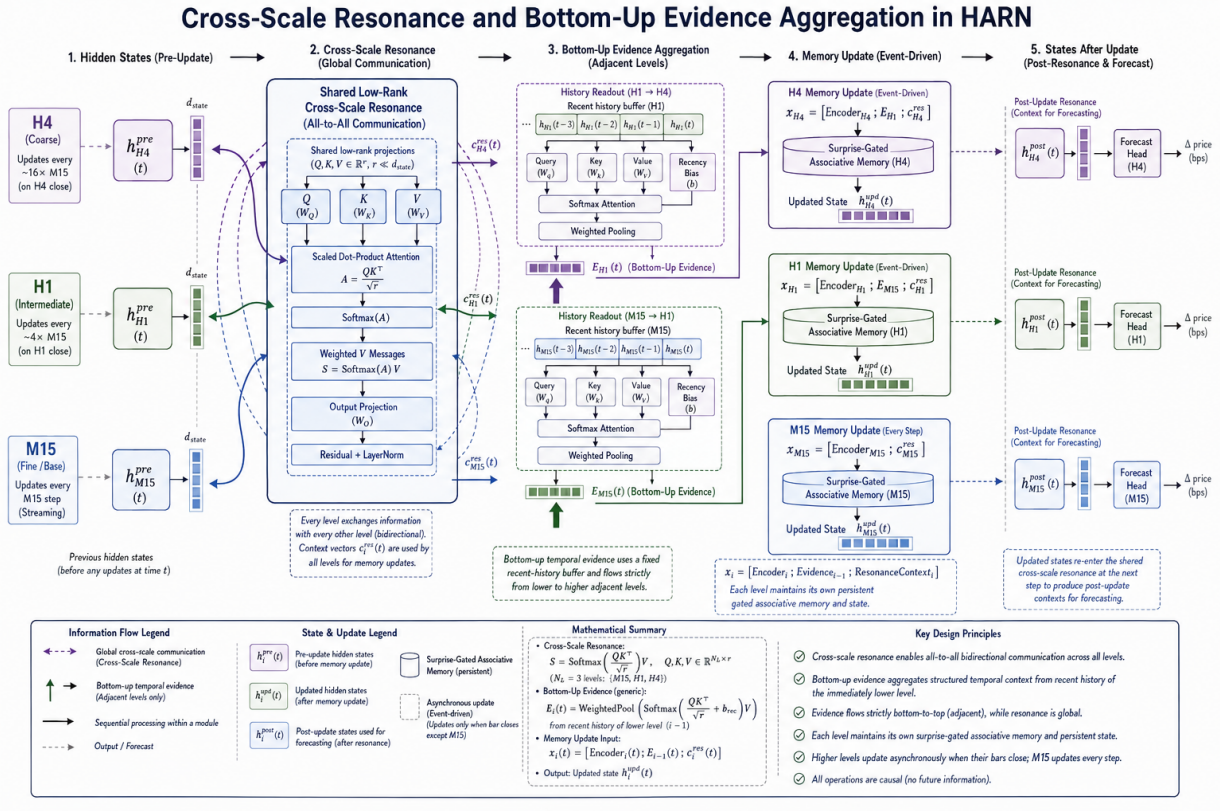


Figure 4. HARN's two cross-level communication paths: all-to-all, contemporaneous resonance (Eq. (22)) and directional, ordered bottom-up evidence (Eq. (23)). This conceptual schematic is not an empirical result.

Forecasting head. The forecasting head maps the cross-scale-informed state to a one-step BPS forecast. It receives the current state, the posterior resonance context, and six anchor features: one-step and two-step close changes, trailing close deviation, trailing range, trailing body, and mean normalized volume. The concatenated input has dimension $72 + 72 + 6 = 150$ in the default configuration. A

two-hidden-layer GELU MLP of width 80 (Hendrycks and Gimpel, 2016) with dropout 0.1 (Srivastava et al., 2014) is combined with a residual linear path from the current state and anchors, allowing the head to fall back to a linear state-dependent function:

$$\hat{y}^k_{t+1} = \mathrm{MLP}\big([r^k_{t+1}; R'^k_{t+1}; a^k_t]\big) + W_{\mathrm{res}}[r^k_{t+1}; a^k_t]. \tag{24}$$

Linear layers use Xavier uniform initialization (Glorot and Bengio, 2010) and convolution layers use Kaiming uniform initialization (He et al., 2015), with zero biases throughout. Both schemes scale the initial weight variance with the layer's fan, and the choice follows the activation used downstream of each layer type. The head outputs a one-step BPS forecast for each configured level. The absolute-price forecast used for evaluation is obtained only afterwards, by inserting the last observed close into the reconstruction stated after Eq. (6).

5 Streaming Semantics and Training

At each anchor event, the implementation selects completed-bar windows, encodes every supplied window, applies prior-state resonance, constructs anchor features, updates the base level, computes lower-level evidence in order, conditionally updates higher levels, applies posterior resonance, and forecasts at every level. The ordered procedure is restated in Section A.3. Candidate transitions are computed for every level at every event, but for each sample only the candidates whose mask entry in Eq. (3) equals one overwrite persistent state (Eq. (4)). Because masking follows a batched forward pass, candidate computation covers the whole batch even when only some streams are active. If activity flags are missing, they default to lock-step updates ($m^k_\tau \equiv \mathbf{1}$ for every level), the special case in which the mechanism reduces to ordinary recurrence. Figure 5 summarizes this ordering.

State handling and loss. States and zero-filled histories are reset at epoch and evaluation boundaries and whenever the batch size changes. This prevents state from one contiguous stream from leaking into an unrelated stream. The production loop detaches state, memory, and history tensors after every optimizer step. With one optimizer step per streaming batch, the effective truncated-backpropagation span is one batch transition (Williams and Peng, 1990). A longer gradient accumulation appears in the demonstration script but is not part of production training. The production loss sums per-level BPS mean-squared error without level normalization. Consequently, hierarchies with different numbers of levels do not share a common loss scale, and a level with larger BPS variance may dominate the gradient:

$$\mathcal{L} = \sum_{k=1}^{L} \frac{1}{B} \sum_{b=1}^{B} (\hat{y}^k_{b,t+1} - y^k_{b,t+1})^2. \tag{25}$$

`StreamingBatcher` divides each chronological split into non-overlapping contiguous streams, discards any remainder that cannot fill the configured partition, and preserves chronological continuity at each batch position. Consecutive batches at a given stream position are therefore consecutive events in time, which is the continuity assumed by the masking and detachment behavior.

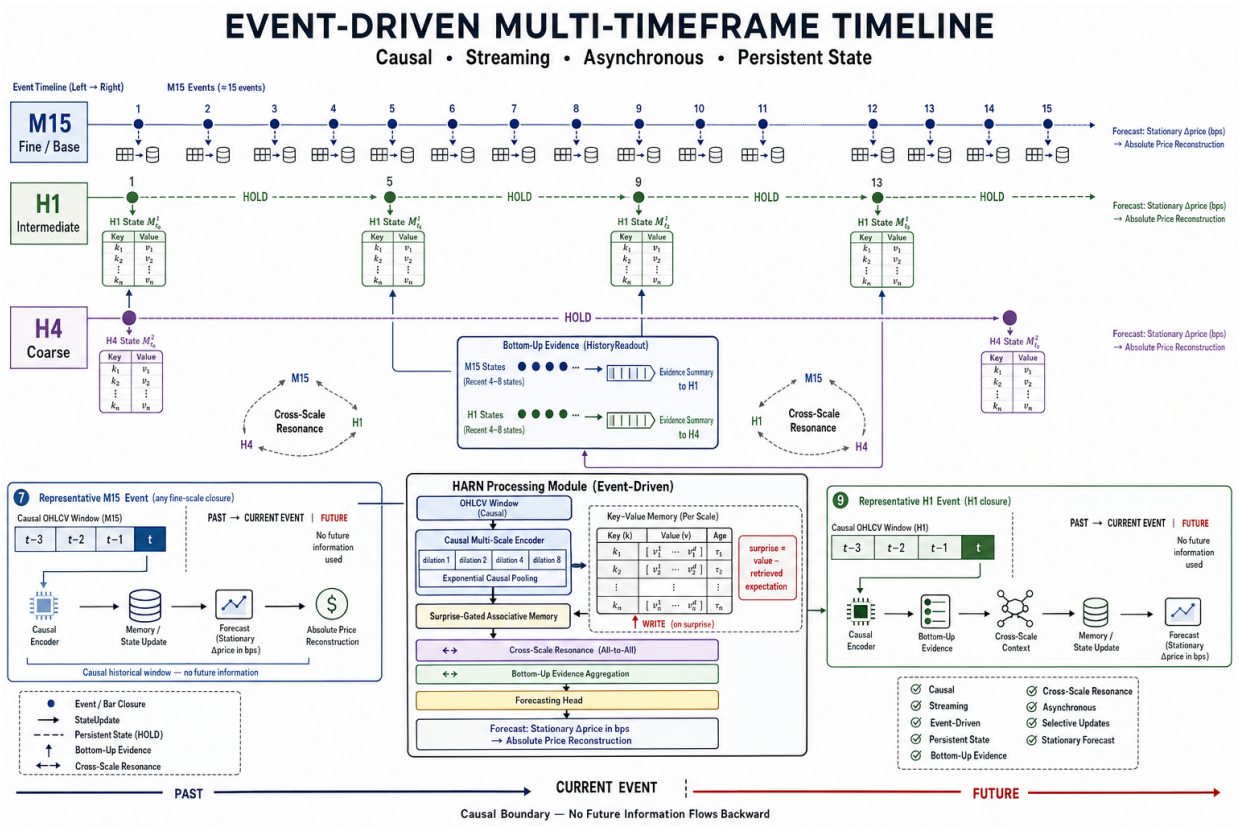


Figure 5. Event-driven multi-timeframe timeline. The anchor stream generates forecast events; a coarser representation is updated only when its completed-bar timestamp equals the anchor timestamp and is otherwise carried forward. This conceptual schematic illustrates the protocol and is not an empirical result.

Training configuration. The production parser's defaults are batch size 256, eight evaluation streams, 50 maximum epochs, AdamW with learning rate 10^{-3} and weight decay 10^{-5} (Loshchilov and Hutter, 2019), gradient clipping at 1.0 (Pascanu et al., 2013), patience 15, seed 42, cosine annealing with $T_{\max}$ = epochs (Loshchilov and Hutter, 2017), and CUDA automatic mixed precision when available (Micikevicius et al., 2018). Its source sets NumPy and PyTorch seeds but does not provide the ablation trainer's full Python-random and deterministic-cuDNN controls. The configuration table accompanying the reported results specifies the same batch size and epoch budget and adds the seeds 42, 151, and 359, whereas the separate ablation trainer defaults to batch size 256 and five epochs. These artifact-level configurations are inventoried in Table 8, and Section 6.2 states which is associated with each reported result. Validation inside the production trainer is stateful over its own contiguous streams, whereas the standalone evaluator uses a single stream and does not reproduce that default exactly.

6 Experimental Setup and Provenance

This section describes the data and comparators. Because several configurations coexist in the available artifacts, it also identifies the configuration associated with each reported result and states the caveat that governs the baseline comparison in later sections.

Per-seed aggregate metric files, the dataset configuration, the complete preprocessing pipeline, the completed-bar alignment and cleaning code, and seed-specific checkpoints are available in the

project repository (Section 12). Raw vendor market data cannot be redistributed, but the preprocessing pipeline is vendor-neutral and can regenerate the dataset from a compatible feed. Per-event predictions and persistence outputs are not included as precomputed files in the current release, although they can be generated from the released checkpoints and preprocessing pipeline. The zero-BPS persistence baseline remains a required rerun, and no values for it are reported here.

6.1 Data, splits, and comparators

The dataset covers four assets with different anchor frequencies and hierarchy depths: AAPL (M15/H1), EURUSD (M15/H1/H4), USDCHF (M15/M30/H1), and XAUUSD (M5/M15). This selection spans equities, foreign-exchange pairs, and gold, but it does not separate asset-specific effects from hierarchy-depth or period effects. Windows contain 32 bars by default unless explicitly overridden. Each sample is aligned to an anchor event and carries the latest completed window for every configured level, together with update indicators and next-bar targets, as formalized in Section 3. Table 1 reports sample counts, periods, and mean update rates.

Table 1. Dataset and split statistics. Each row gives, for one asset and chronological split, the hierarchy, anchor timeframe, number of anchor-event observations, period, span in days, and mean update rate (the event-update rate averaged across levels). Per-level update rates are listed beneath the table.

Asset	Split	Hierarchy	Anchor	Observations	Period	Span (days)	Mean update
AAPL	Train	M15,H1	M15	43,331	2017-02-07 → 2023-09-27	2,423.1	0.616
AAPL	Val	M15,H1	M15	9,285	2023-09-27 → 2025-03-06	525.8	0.615
AAPL	Test	M15,H1	M15	9,286	2025-03-06 → 2026-08-10	522.2	0.615
EURUSD	Train	M15,H1,H4	M15	69,967	2022-07-26 → 2025-05-15	1,024.2	0.438
EURUSD	Val	M15,H1,H4	M15	14,992	2025-05-15 → 2025-12-19	218.2	0.438
EURUSD	Test	M15,H1,H4	M15	14,994	2025-12-19 → 2026-07-29	221.8	0.437
USDCHF	Train	M15,M30,H1	M15	69,975	2022-08-05 → 2025-05-28	1,026.8	0.583
USDCHF	Val	M15,M30,H1	M15	14,994	2025-05-28 → 2026-01-04	221.8	0.583
USDCHF	Test	M15,M30,H1	M15	14,996	2026-01-04 → 2026-08-11	218.2	0.583
XAUUSD	Train	M5,M15	M5	69,976	2025-03-13 → 2026-03-10	361.6	0.667
XAUUSD	Val	M5,M15	M5	14,994	2026-03-10 → 2026-05-26	77.4	0.667
XAUUSD	Test	M5,M15	M5	14,996	2026-05-26 → 2026-08-11	76.7	0.667

Per-level update rates (finest to coarsest): AAPL M15/H1 = 1.00/0.232; EURUSD M15/H1/H4 = 1.00/0.250/0.063; USDCHF M15/M30/H1 = 1.00/0.500/0.250; XAUUSD M5/M15 = 1.00/0.333.

The per-level update rates in Table 1 show the expected step pattern: exactly 1.0 at the anchor level and decreasing at each coarser level, roughly in line with the ratio of bar durations. For example, USDCHF's H1 level closes once per four completed M15 bars, giving a rate of 0.250. This is a sanity check on the completed-bar alignment of Section 3, consistent with the update indicator being computed correctly upstream of the masked transition; it is not an independent empirical claim. The counts, dates, and update rates are transcribed from the dataset configuration. The expected input format and cleaning procedure are described in Section 12.

PatchTST uses $d_{\text{model}} = 128$, $d_{ff} = 256$, two layers, eight heads, patch length 16, stride 8, disabled RevIN and decomposition, and horizon one (Nie et al., 2023). TimeXer shares these principal dimensions, patches the close price, and treats the remaining OHLCV channels as exogenous variables (Wang et al., 2024). Both are single-timeframe architectures. They do not natively support multi-timeframe, event-driven state, so they are trained on the anchor timeframe, which is their native

configuration. This places the two baselines within the single-timeframe architectural comparisons used in controlled multi-asset forecasting benchmarks (Saidd, 2026a), whereas HARN is a multi-timeframe, event-driven model and is evaluated with all configured payload levels.

The comparison in Section 7 is therefore a *descriptive, scope-specific comparison of complete pipelines*. As Table 2 shows, HARN receives every configured payload level and trains with stationary BPS targets, stateful contiguous streams, cosine scheduling, patience 15, and CUDA AMP when available. The baselines train on normalized absolute-close targets with constant learning rates, patience 5, stateless shuffled batches, and no AMP. This target difference is substantive: by Eq. (7), HARN's price error is a price-scaled BPS error anchored at the last observed close, whereas the baselines regress a normalized absolute close. The reported gap therefore does not isolate hierarchy, memory, resonance, target representation, training protocol, tuning effort, or architecture. This is the governing caveat for the baseline comparison.

Table 2. Scope and training differences between HARN and the single-timeframe baseline trainers. PatchTST and TimeXer are trained on the anchor timeframe, their native configuration, whereas HARN is evaluated as a multi-timeframe model. These differences make the comparison in Table 4 descriptive: it does not isolate architecture, target definition, or training recipe.

Dimension	HARN	PatchTST	TimeXer
Scope	Multi-timeframe, event-driven	Single-timeframe, anchor-only	Single-timeframe, anchor-only
Input	5 OHLCV, all payload levels	5 OHLCV, anchor only	5 OHLCV, anchor only
Lookback / horizon	32 per level / 1	32 / 1	32 / 1
Training target	Stationary BPS delta	Normalized absolute-close target	Normalized absolute-close target
Batch/protocol	256 (reported recipe); stateful contiguous streams	256 shuffled; stateless	256 shuffled; stateless
Schedule; patience	Cosine; 15	Constant; 5	Constant; 5
AMP	CUDA AMP if available	No	No
Seed	Code default 42; aggregate results over 42, 151, 359	Configuration table records 42; per-seed CSVs cover 42, 151, 359	Configuration table records 42; per-seed CSVs cover 42, 151, 359
Selection	Best validation loss	Best validation loss	Best validation loss

6.2 Which configuration produced which reported result

Because the available artifacts contain several configurations (Table 8), Table 3 records, for each reported item, the configuration that the manuscript's own evidence associates with it and what cannot be determined from the current materials. Where the materials are insufficient, the ambiguity is stated rather than resolved by inference.

Table 3. Provenance of each reported item, based on statements in this manuscript and the repository artifacts. For each item, the table gives the associated configuration and what is and is not determined from the current materials.

Reported item	Associated configuration	Determined / not determined
Dataset table (Table 1)	Dataset configuration; `dataset_config` is available in the repository.	Values are transcribed. Raw vendor data cannot be redistributed, but the dataset can be regenerated from a compatible vendor feed with the vendor-neutral preprocessing pipeline.
HARN rows, Panels A and B (mean ± SD over three seeds)	Reported recipe: batch size 256, 50 epochs, AdamW (10^{-3}, 10^{-5}), clipping 1.0, cosine schedule, patience 15, CUDA AMP if available; seeds 42, 151, 359.	The recipe coincides with the production defaults except for the seed list (Table 8). Seed-specific checkpoints and per-seed aggregate metric CSVs are available, and per-event predictions can be generated from them with the preprocessing pipeline. The standalone evaluator uses one stream, whereas the recipe describes eight; the stream count behind the metrics should be read from the available configuration and checkpoint metadata. Whether Panels A and B came from the same checkpoints is documented in repository metadata where available and is otherwise unresolved.
PatchTST and TimeXer rows, Panel A	Baseline configuration of Table 2: single-timeframe, anchor-only, dimensions and patching as in Section 6, batch 256 shuffled, constant learning rate, patience 5, no AMP.	Per-seed aggregate metrics are available in the repository, and per-event predictions can be generated from the available pipeline and checkpoints. In Panel A, HARN and the baselines are reported at the anchor timeframe of each asset (M15 for AAPL, EURUSD, and USDCHF; M5 for XAUUSD).
Ablation variants A0–A3 (Table 5)	Ablation trainer defaults: batch 256, five epochs, seed 42, cosine schedule, patience 15, with configuration and seed retained in its checkpoint format.	The available evidence does not establish that these defaults were used for the table or identify repeated runs. A0 is full HARN within the ablation pipeline, but its values differ from the main HARN means and are not verified replicates; the reason is undetermined (Section 8).
Figures 6 and 7	Evaluation visualizations from the available outputs.	The captions do not identify asset, timeframe, seed, checkpoint, or split; the underlying predictions can be regenerated from the available checkpoints and preprocessing pipeline.
Audit table (Table 6)	Static inspection of the source code.	Not an experiment (Section 9).

7 Metrics and Reported Results

For price errors $e_i = \hat{P}_i - P_i$, the evaluators report four point-forecast accuracy metrics that give both scale-dependent and scale-normalized views, consistent with the recommendation that no single measure suffices across series of different scales (Hyndman and Koehler, 2006):

$$\mathrm{MAE} = n^{-1} \sum_i |e_i|, \qquad \mathrm{RMSE} = \sqrt{n^{-1} \sum_i e_i^2}, \tag{26}$$

$$\mathrm{sMAPE} = 100\, n^{-1} \sum_i \frac{2|e_i|}{|P_i| + |\hat{P}_i|}, \qquad \mathrm{MASE} = \frac{\mathrm{MAE}}{\mathrm{mean}_j\, |y_{j+1}^{\mathrm{train}} - y_j^{\mathrm{train}}|}. \tag{27}$$

MAE and RMSE are in the asset's native price units, and sMAPE is a symmetric percentage error. The evaluator's MASE divides the reconstructed-price MAE by the mean absolute first difference of the stored training target-event sequence. Because higher-level target values can repeat while a coarser bar is carried forward, this denominator is an implementation-defined scale rather than necessarily the chronological close-series denominator of the standard MASE definition (Hyndman and Koehler, 2006). The evaluators do not emit a target-space metric, so all reported values are computed on reconstructed absolute price. The natural persistence comparator predicts $\hat{y}_i = 0$ and therefore $\hat{P}_i = P_i^{\mathrm{last}}$ at each forecast origin. No persistence output is included as a precomputed file in the current release, and

this comparator is a required rerun, not a missing value that can be inferred from the aggregate price metrics.

Table 4 reports the results. All values are transcribed as computed by the evaluators. The ± figures are aggregate variability across three seeds, whose per-seed outputs are given in Section A.5, and Section 6.2 records the configuration associated with each panel.

Table 4. Forecasting results on the test split, reported as mean ± standard deviation across three seeds. Lower is better for every metric, and all metrics are computed on reconstructed absolute price. Panel A is a descriptive, scope-specific comparison between HARN (multi-timeframe, event-driven, BPS target, stateful training) and single-timeframe PatchTST and TimeXer trained on the anchor timeframe; it does not isolate architecture, target definition, or training recipe. Panel B reports HARN at further timeframes, for which no baseline was run; its XAUUSD M5 row repeats the Panel A entry. Values are reported as computed by the evaluators.

Panel/asset	Model/TF	MAE	RMSE	sMAPE	MASE
Panel A: HARN (multi-timeframe) versus single-timeframe anchor-only baselines (descriptive)					
AAPL	HARN/M15	0.507696±0.000509	0.933489±0.000420	0.204947±0.000209	1.327584±0.001331
	PatchTST/M15	2.074401±0.575730	2.702157±0.559581	0.794647±0.219045	5.424392±1.505489
	TimeXer/M15	2.881886±0.811630	3.613453±0.848218	1.115299±0.292503	7.535901±2.122348
EURUSD	HARN/M15	0.000306±0.000002	0.000467±0.000002	0.026311±0.000188	0.865688±0.006210
	PatchTST/M15	0.000759±0.000275	0.000967±0.000305	0.065011±0.023455	2.144210±0.777650
	TimeXer/M15	0.010838±0.000347	0.013051±0.000521	0.930031±0.029837	30.626894±0.979317
USDCHF	HARN/M15	0.000263±0.000000	0.000391±0.000000	0.033284±0.000014	0.800131±0.000345
	PatchTST/M15	0.000638±0.000100	0.000864±0.000106	0.080930±0.012893	1.941151±0.304746
	TimeXer/M15	0.014144±0.000505	0.015651±0.000639	1.782636±0.063131	43.035375±1.536711
XAUUSD	HARN/M5	2.552111±0.001505	3.844300±0.001443	0.061098±0.000036	1.136012±0.000670
	PatchTST/M5	9.563464±1.040070	12.678673±0.945826	0.228356±0.025547	4.256949±0.462962
	TimeXer/M5	15.297477±1.745931	19.094934±2.925531	0.361264±0.038120	6.809308±0.777160
Panel B: HARN forecasts at further timeframes (no baseline available)					
AAPL	H1	1.059066±0.000930	1.800225±0.000686	0.426064±0.000297	4.817277±0.004229
EURUSD	H1	0.000616±0.000002	0.000906±0.000002	0.052931±0.000188	3.463016±0.012255
EURUSD	H4	0.001227±0.000002	0.001745±0.000001	0.105431±0.000127	13.543420±0.016580
USDCHF	M30	0.000373±0.000001	0.000543±0.000000	0.047208±0.000065	1.607638±0.002219
USDCHF	H1	0.000526±0.000001	0.000763±0.000000	0.066632±0.000122	3.232735±0.005914
XAUUSD	M5	2.552111±0.001505	3.844300±0.001443	0.061098±0.000036	1.136012±0.000670

7.1 Baseline comparison and further timeframes

In Panel A of Table 4, HARN's price errors are numerically lower than those of both baselines for all four assets at the anchor timeframe: M15 for AAPL, EURUSD, and USDCHF, and M5 for XAUUSD. The XAUUSD HARN entries equal the means of the per-seed M5 values in Table 9. As Section 6 explains, this is a statement about the configured pipelines (Table 2), not an architectural result. No reported run varies the available timeframes, target representation, or training protocol one factor at a time. The results therefore do not identify whether a numerical difference arises from additional timeframes, the BPS target, stateful streaming, tuning effort, or architecture. Three further points guide interpretation.

Missing persistence baseline. The reported results contain no persistence (last-value, that is, zero-BPS) or other random-walk baseline. Because HARN's price error is a price-scaled BPS error anchored at the last close (Eq. (7)), such a reference is the natural yardstick, and its absence leaves the results without an interpretability baseline. The MASE column does not fill this role. Its denominator

is an implementation-defined training-sample scale, and HARN's own MASE values range from below one (EURUSD and USDCHF at the anchor) to above one (the AAPL and XAUUSD anchor rows and all higher levels), so they do not settle whether HARN improves on persistence over the test period.

Omitted percentage reductions. The original results also expressed HARN's advantage over PatchTST as percentage reductions, which are omitted here. They restate Panel A as an effect-size-like quantity for a scope-specific comparison and so invite an attribution to architecture that the design cannot support. They are also ratios of aggregate means without uncertainty, their aggregation method is undocumented, and a few entries differ by about 0.1 percentage points from the reduction implied by the rounded tabulated means. Nothing has been recomputed or replaced.

Further timeframes. Panel B reports HARN's forecasts at the further levels of each hierarchy, which have different update rates (Table 1). It documents forecasts at several resolutions and does not show superiority, cross-level forecast coherence, or benefit from the hierarchy. Its XAUUSD M5 row repeats the anchor entry of Panel A, and per-seed results for the non-anchor XAUUSD M15 level appear in Table 9. MASE denominators at higher levels use repeated targets, so MASE values are not comparable across timeframes.

7.2 Cross-seed variance and statistical status

The reported HARN standard deviations across seeds are much smaller than those of the baselines. For AAPL MAE, HARN's standard deviation is about 0.1% of its mean, compared with about 28% for both PatchTST and TimeXer; for EURUSD MAE, the corresponding figures are below 1% for HARN and about 36% for PatchTST. Several HARN entries round to ±0.000000 at the reported precision, including USDCHF MAE and RMSE in Panel A and RMSE at M30 and H1 in Panel B. The pattern appears across metrics and panels.

We regard this as an unresolved variance observation: the available evidence does not establish its cause, and we do not offer one. Seed-specific checkpoints and per-seed aggregate metrics are available (Section A.5), but per-seed training trajectories and per-event predictions are not included as precomputed files in the current release. Seed independence therefore cannot be verified from the CSVs and checkpoints alone; demonstrating that the three seeds (42, 151, 359) produced independent training trajectories requires inspecting the seed-specific checkpoints and their configuration metadata. The small HARN spread should not be read as evidence of robustness, and the baselines' larger spread should not be read as a property of those architectures, given the differing protocols. Interpreting either requires the per-event outputs and the verification identified here as necessary follow-up work.

Figures 6 and 7 are empirical visualizations taken from the evaluation outputs, in contrast with the conceptual architecture and timeline figures. Their captions do not identify the asset, timeframe, seed, or split shown, and the underlying predictions are not included as precomputed files in the current release, so the figures provide qualitative context only and no statistical comparison.

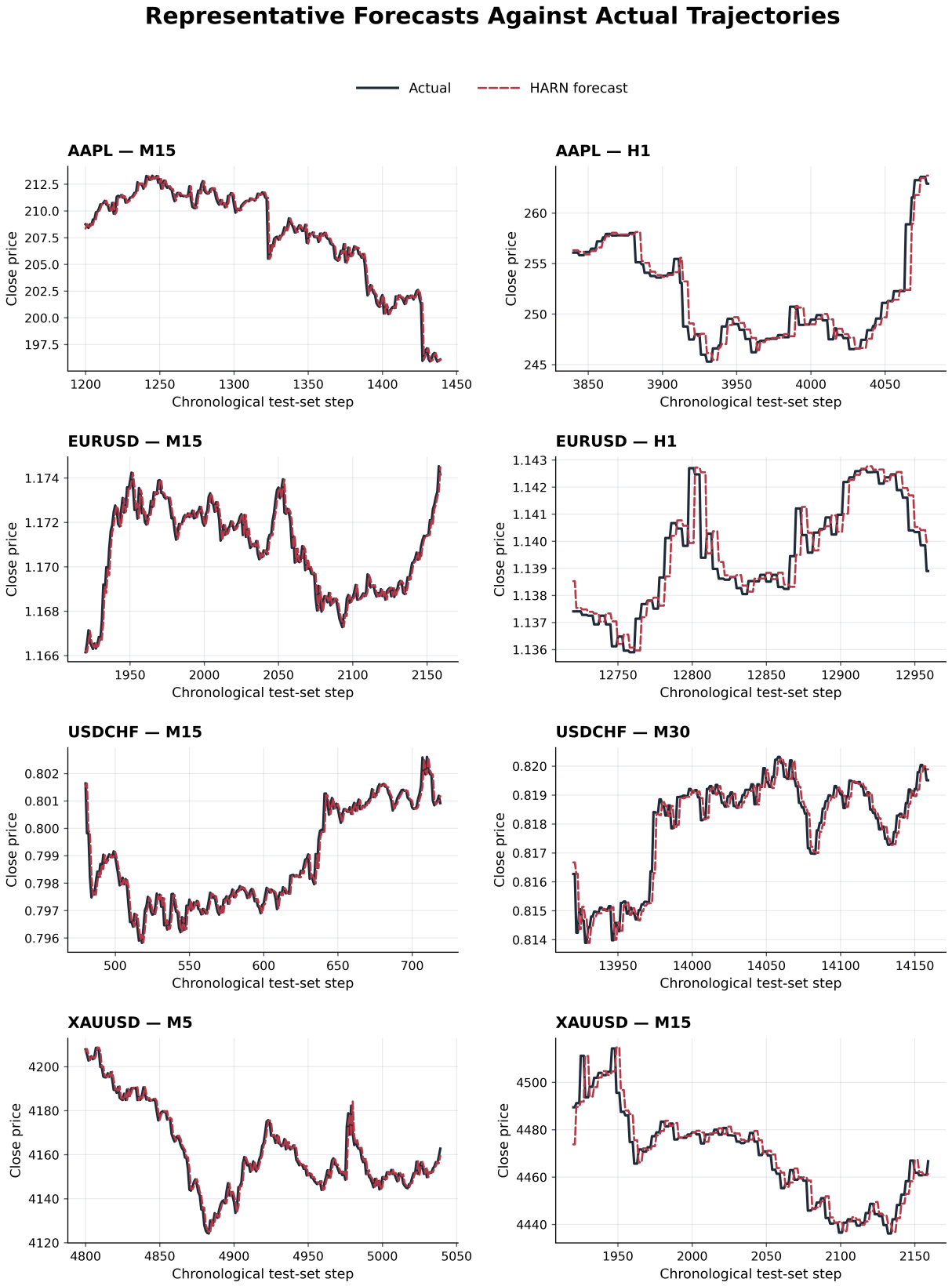


Figure 6. Representative forecasts against actual trajectories, taken from the evaluation outputs. The asset, timeframe, seed, and split are not identified in the available materials, so the figure gives qualitative context only.

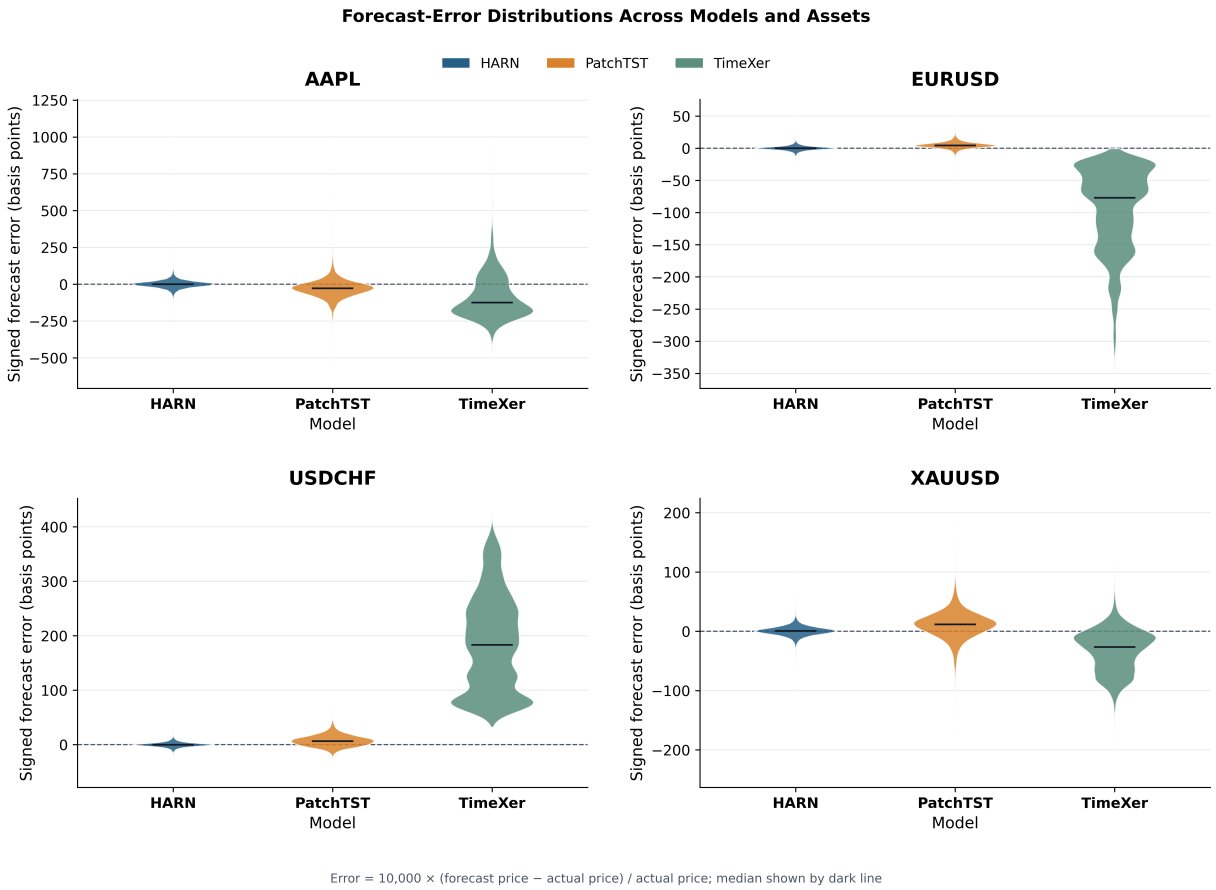


Figure 7. Representative forecast-error distributions across models and assets, taken from the evaluation outputs. The provenance limits are the same as for Figure 6, and the figure gives qualitative context only.

A standard deviation over three aggregate seed scores is not a confidence interval for the temporal forecast-error process, does not capture dependence across forecast origins, and cannot recover the paired model differences required by a formal predictive-accuracy test (Diebold and Mariano, 1995; Giacomini and White, 2006). No confidence intervals, paired tests, Diebold–Mariano tests, Wilcoxon tests, effect sizes, or multiplicity corrections were computed, and none are introduced here. Section 10 identifies these analyses as proposed work.

8 Ablation Variants: Single-Run Observations

The ablation study asks a narrower question than the baseline comparison: how do HARN's own reported values change when one named internal mechanism is replaced or bypassed, with the surrounding streaming pipeline and tensor interfaces held fixed? The variants share HARN's interfaces and pipeline but not its parameterization or parameter count, so they are component-replacement interventions rather than matched counterfactuals. A0 is full HARN. A1 replaces the associative cell of Section 4.2 with a gated recurrent-style update while carrying the memory tensor unchanged. A2 replaces resonance (Eq. (22)) with the identity map while retaining bottom-up evidence, so it removes only the all-to-all path and not all cross-level communication. A3 replaces the history readout (Eq. (23)) with zero evidence while preserving the interface and history maintenance. Each intervention can alter capacity and optimization dynamics in addition to the named mechanism: A1 changes the cell's

parameterization, A2 removes learnable resonance parameters, and A3 removes learned evidence content.

Table 5 covers only AAPL and EURUSD and is rounded to six decimals. Each cell is a single aggregate value with no uncertainty estimate, and the available evidence does not document repeated runs, so the entries are treated as single-run observations rather than statistically established sensitivity effects. For metric m, the relative degradation is defined as

$$\Delta_i = \frac{m(A_i) - m(A_0)}{|m(A_0)|}, \tag{28}$$

where a positive value indicates higher error than the full model.

Table 5. Ablation variants for AAPL and EURUSD only. Each cell is a single aggregate value without per-seed uncertainty, and lower is better. A0 is full HARN; A1 replaces the associative memory with a gated recurrent update; A2 replaces resonance with the identity map; A3 zeros the bottom-up evidence readout. Configuration provenance is given in Table 3.

Asset	TF	Variant	MAE	RMSE	sMAPE	MASE
AAPL	H1	A0	1.055134	1.801484	0.424678	4.799395
AAPL	H1	A1	1.056989	1.798886	0.425441	4.807829
AAPL	H1	A2	1.060339	1.803577	0.426672	4.823069
AAPL	H1	A3	1.058882	1.796596	0.426056	4.816440
AAPL	M15	A0	0.506070	0.932352	0.204299	1.326365
AAPL	M15	A1	0.507230	0.933322	0.204760	1.323333
AAPL	M15	A2	0.507286	0.933827	0.204801	1.326513
AAPL	M15	A3	0.508024	0.934081	0.205084	1.328443
EURUSD	H1	A0	0.000610	0.000902	0.052438	3.469651
EURUSD	H1	A1	0.000617	0.000907	0.053033	3.430735
EURUSD	H1	A2	0.000614	0.000903	0.052699	3.447709
EURUSD	H1	A3	0.000613	0.000904	0.052646	3.444383
EURUSD	H4	A0	0.001226	0.001742	0.105436	13.534480
EURUSD	H4	A1	0.001231	0.001747	0.105728	13.581712
EURUSD	H4	A2	0.001229	0.001746	0.105552	13.559399
EURUSD	H4	A3	0.001227	0.001743	0.105364	13.544075
EURUSD	M15	A0	0.000305	0.000465	0.026176	0.861235
EURUSD	M15	A1	0.000306	0.000466	0.026240	0.863367
EURUSD	M15	A2	0.000305	0.000465	0.026183	0.861477
EURUSD	M15	A3	0.000306	0.000466	0.026250	0.863671

Two features of Table 5 limit what it can show. First, the differences are small and non-monotonic. A0 is not uniformly best: A1 and A3 have lower AAPL H1 RMSE than A0, and A1–A3 all have lower EURUSD H1 MASE. A variant scoring below A0 on one metric does not by itself indicate that the removed mechanism is harmful, given the capacity and optimization differences described above. Second, A0 does not coincide with the corresponding HARN means in Table 4. For AAPL M15, the A0 MAE is 0.506070 against a Panel A mean of 0.507696, and for AAPL H1 it is 1.055134 against a Panel B mean of 1.059066. These gaps are of similar size to the gaps between A0 and A1–A3, and the manuscript does not establish whether A0 is a rerun, a different epoch budget, or a different evaluation path. Variant differences of this size therefore cannot currently be separated from unexplained run-to-run or protocol differences. A measured reading is that no single component

dominates these single-run observations, and that firmer statements would require the multi-seed, capacity-matched experiments identified as proposed work. Figure 8 plots Δ_i and is descriptive only.

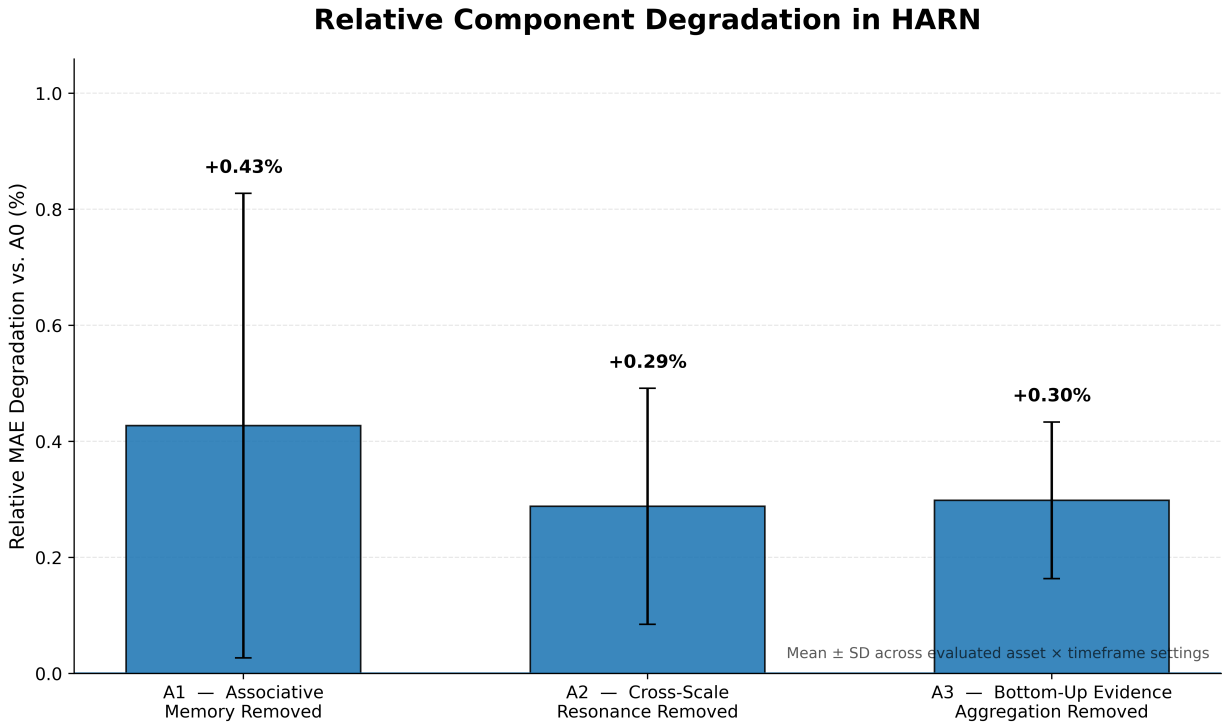


Figure 8. Relative component degradation Δ_i (Eq. (28)) for each ablation variant, asset, and timeframe, computed from Table 5. The plot summarizes single aggregate values; it is descriptive and carries no uncertainty estimate.

9 Code-Level Causality Audit

This section records a static review of the source code against the event-level causal contract defined in Section 3. It is a code-level causal-consistency audit and *not* an empirical test: no experiment was conducted in which bars after τ were perturbed, shuffled, or withheld to verify that forecasts remain unchanged. Code review also cannot detect upstream problems in the data itself, such as source-level bar construction, timestamp conventions, clock synchronization, or price adjustment. An empirical check of this kind remains proposed work. Table 6 summarizes the audit of chronological splitting, completed-bar alignment, next-bar targets, training-only statistics, left padding, shifted derived features, state masking, reset and detachment behavior, contiguous streams, cleaning, and validation-based checkpoint selection, following the implementation-level considerations relevant to leakage and validation in the forecasting literature (Bergmeir et al., 2018; Kaufman et al., 2012; Tashman, 2000). The findings indicate consistency with the specified future-event information-ordering contract under the implementation's event ordering. Reproducibility is supported by the artifacts listed in Section 12, subject to the raw-data redistribution constraint.

Three qualifications apply. First, the lower-level state is appended before the higher-level evidence is read within the same event. The evidence path is thus ordered with respect to future events but is not strictly past-event-only within an event; this is the same-event evidence-ordering qualification of Section 3, recorded in row 9 of Table 6. Second, inactive levels are state-preserving but not

computation-free: their windows and anchors are processed at every event, and only the persistent overwrite is masked (Eq. (2)). Third, the exact-timestamp-equality rule underlying u_τ^k is a data-interface assumption. Feeds with different timestamp conventions, clock skew, or asynchronous bar-close reporting would require an explicit alignment policy before the same audit could be applied unchanged.

Table 6. Condensed code-level causal-consistency audit from a static review of the source code; it is not an empirical causality test. "Consistent (qualified)" denotes consistency with the event-level information-ordering contract, subject to the same-event evidence-ordering qualification of Section 3.

#	Dimension	Mechanism observed in the code	Finding
1	Splitting	Chronological 70/15/15 train–validation–test slices	Consistent
2	Alignment	Latest completed timestamp no later than the anchor; exact timestamp equality determines an update	Consistent
3	Windows/ targets	Window ends at the selected completed bar; target is the next completed bar	Consistent
4	Statistics	Training-only feature and target statistics are reused during evaluation	Consistent
5	Local causality	Left-only padding and shifted previous-close features avoid dependence on future observations	Consistent
6	Resonance	Only states available at the same event are mixed	Consistent
7	Memory	Current input, previous state, and bounded residual-driven write determine memory updates	Consistent
8	Masked updates	Inactive higher-level persistent states and memory are retained rather than overwritten	Consistent
9	History	Current lower-level read is appended before higher-level readout	Consistent (qualified)
10	Lifecycle	States are reset at boundaries and states and histories are detached after steps	Consistent
11	Streaming	Non-overlapping chronological streams preserve batch-position continuity	Consistent
12	Cleaning	Invalid rows are removed, duplicates are resolved, and timestamps are sorted chronologically	Consistent
13	Selection	Validation loss selects the checkpoint before test evaluation	Consistent

#	Dimension	Mechanism observed in the code	Finding
14	Test isolation	Test observations are not used for training or validation	Consistent

10 Discussion and Limitations

What the evidence supports. HARN is an event-driven, multi-timeframe architecture that maintains a compact state for each configured resolution and updates that state when new information becomes available at the corresponding resolution. Its principal mechanisms are residual-driven associative memory, cross-scale resonance, and ordered history readout. Together, they retain, exchange, and read information across asynchronously completing timeframes. The experiments characterize this design under the specified completed-bar protocol, and the code-level audit finds the implementation consistent with the event-driven alignment rules used in the experiments. Under the reported configuration, HARN achieves lower reconstructed-price errors than the accompanying single-timeframe, anchor-only baselines on the evaluated datasets. That comparison remains descriptive, however, because the models differ in information access, target formulation, and training procedure. The ablations show only small changes when individual components are replaced, but their single-run design does not establish independent component effects. The results therefore support the viability of the design in the evaluated setting; they do not establish superiority over alternative architectures or attribute the observed differences to any one design choice.

Limitations. The main limitations concern comparison scope, reference baselines, ablation strength, generalization, computational evaluation, and design assumptions.

- *Comparison scope.* The baseline comparison does not isolate architectural effects from differences in information access, target formulation, or training procedure, and it is therefore not a controlled architectural comparison.
- *Reference baselines and statistics.* No persistence baseline, confidence intervals, or paired predictive-accuracy tests are reported, and the unusually small cross-seed spread of HARN remains unexplained.
- *Ablation strength.* The ablations are single-run experiments on a subset of assets and are not parameter- or compute-matched. They provide component-level observations but not a statistically robust attribution of performance to individual mechanisms.
- *Generalization.* The study covers four assets and specific evaluation periods. The available evidence does not establish how performance changes across a broader range of assets, market conditions, hierarchy depths, or time periods.
- *Computational evaluation.* The study does not report systematic measurements of inference latency, peak memory, throughput, or energy consumption, so the practical cost of maintaining and updating multiple resolution-specific states remains incompletely characterized.
- *Design assumptions.* The evidence-readout coefficients are hand-set and unvalidated, and the update indicator relies on exact timestamp equality under the chosen bar-construction protocol.

Proposed experiments. The following studies have not been performed and constitute proposed work: (i) evaluation of the zero-BPS persistence baseline; (ii) generation of per-event predictions from the available checkpoints to enable confidence intervals and paired tests such as Diebold–Mariano and conditional predictive-ability tests; (iii) verification of seed independence from checkpoint metadata and training trajectories; (iv) controlled comparisons that vary the available timeframes, target representation, and training protocol one factor at a time, together with matched multi-timeframe or event-driven baselines; (v) multi-seed, capacity-matched ablations, including sensitivity to the hand-set evidence coefficients; (vi) an empirical perturbation test in which bars after the anchor time are altered or withheld; (vii) systematic measurement of latency, memory, and throughput; and (viii) archiving generated outputs with command lines, source hashes, data checksums, and software and device versions.

11 Conclusion

This paper specifies HARN as an event-driven, stateful multi-timeframe forecaster combining causal local encoding, gated matrix memory, all-to-all cross-scale resonance, directional bottom-up evidence, masked asynchronous updates, and BPS reconstruction. Its central contribution is the completed-bar update protocol, formalized in scalar and batched form and examined through a code-level event-causality audit. The reported results show numerically lower absolute-price error for HARN than for the accompanying PatchTST and TimeXer runs. Those baselines are single-timeframe models trained on the anchor timeframe in their native configuration, whereas HARN is evaluated as a multi-timeframe, event-driven model. The comparison is therefore descriptive and scope-specific; it does not isolate architecture, target definition, or training recipe. The single-run ablations identify no dominant component. Whether the reported differences reflect multi-timeframe inputs, target representation, training procedure, or their interaction remains open. Addressing this question requires the broader comparisons, persistence evaluation, per-event predictions, and paired statistical analyses described in Section 10.

12 Data and Code Availability

Repository artifacts. The project repository contains all source code examined in this study, including the HARN implementation; data-cleaning and completed-bar alignment code; the preprocessing pipeline; dataset configuration; the production training script; a standalone evaluator; an ablation trainer; and the PatchTST and TimeXer trainers. It also includes model checkpoints for each seed and per-seed result CSV files. The candidate public URL is https://github.com/NabeelAhmadSaidd/HARN.

Data sharing and vendor-agnostic acquisition. The raw market data used in the reported runs cannot be redistributed because the author does not have permission to share the vendor-provided data. The preprocessing pipeline is vendor-neutral and can be applied to compatible data from another market-data vendor. The expected input is one row per completed bar and timeframe, with columns for timestamp, open, high, low, close, and volume. Timestamps must be parseable and use a consistent

timezone and session convention; bar construction must follow the configured per-level timeframes (for example, M5, M15, M30, H1, and H4). Cleaning parses timestamps, removes invalid rows, retains the last occurrence of duplicate timestamps, and sorts rows in ascending order. Completed-bar alignment selects the latest level-k bar whose timestamp is no later than the anchor timestamp, and the update indicator is set by exact timestamp equality. Splits are chronological 70%/15%/15%, and feature and target statistics are computed on training samples only. A compatible vendor feed with the same schema and bar-construction rules can therefore regenerate the dataset with the provided pipeline.

Checkpoints, evaluator, and generated outputs. The production trainer writes seed-specific checkpoints. The standalone evaluator reconstructs missing configuration fields from defaults and tensor shapes and evaluates on a single stream rather than the trainer's eight-stream default. Ablation checkpoints retain configuration metadata but are not parameter-count matched across variants. Per-event predictions and persistence outputs are not included as precomputed files in the current per-seed CSV release, but they can be generated from the available checkpoints and preprocessing pipeline with a compatible vendor feed. Archiving those generated outputs alongside the exact command line, source hash, data checksums, Python, PyTorch, and CUDA versions, and device details remains proposed work.

Command templates. The source-level command templates are as follows: production HARN training with `python -m src.models.train_harn` and `-asset ASSET`; standalone evaluation with `python -m src.models.evaluate_harn`, `-asset ASSET`, and `-seed SEED`; and ablation training with `python -m src.ablation.train_harn_ablations`, `-asset ASSET`, and `-variant A0` through `A3`. These templates are documented commands, not recovered historical commands for the reported tables.

Conflicts of Interest

The author declares that there are no conflicts of interest associated with this work. The research was conducted independently, with no financial, commercial, institutional, or personal interests that could reasonably be considered to have influenced the design, implementation, evaluation, or interpretation of the study.

No external organization provided funding, directed the experimental methodology, or exercised control over the reported results. The work is presented solely for research purposes and does not constitute investment advice, a recommendation to trade any financial instrument, or an endorsement of any particular financial product or service.

References

Ba, J. et al. (2016). "Using Fast Weights to Attend to the Recent Past". In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., pp. 4331–4339. DOI: 10.48550/arXiv.1610.06258.

Ba, J. L., J. R. Kiros, and G. E. Hinton (2016). "Layer Normalization". In: *arXiv preprint arXiv:1607.06450*. DOI: 10.48550/arXiv.1607.06450.

Bergmeir, C., R. J. Hyndman, and B. Koo (2018). "A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction". In: *Computational Statistics & Data Analysis* 120, pp. 70–83. DOI: 10.1016/j.csda.2017.11.003.

Bollerslev, T. (1986). "Generalized Autoregressive Conditional Heteroskedasticity". In: *Journal of Econometrics* 31.3, pp. 307–327. DOI: 10.1016/0304-4076(86)90063-1.

Chollet, F. (2017). "Xception: Deep Learning with Depthwise Separable Convolutions". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 1800–1807. DOI: 10.1109/CVPR.2017.195.

Cont, R. (2001). "Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues". In: *Quantitative Finance* 1.2, pp. 223–236. DOI: 10.1088/1469-7688/1/2/304.

Diebold, F. X. and R. S. Mariano (1995). "Comparing Predictive Accuracy". In: *Journal of Business & Economic Statistics* 13.3, pp. 253–263. DOI: 10.1080/07350015.1995.10524599.

Engle, R. F. (1982). "Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation". In: *Econometrica* 50.4, pp. 987–1007. DOI: 10.2307/1912773.

Fama, E. F. (1970). "Efficient Capital Markets: A Review of Theory and Empirical Work". In: *The Journal of Finance* 25.2, pp. 383–417. DOI: 10.1111/j.1540-6261.1970.tb00518.x.

Giacomini, R. and H. White (2006). "Tests of Conditional Predictive Ability". In: *Econometrica* 74.6, pp. 1545–1578. DOI: 10.1111/j.1468-0262.2006.00718.x.

Glorot, X. and Y. Bengio (2010). "Understanding the Difficulty of Training Deep Feedforward Neural Networks". In: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 9. Proceedings of Machine Learning Research. PMLR, pp. 249–256.

Gneiting, T. and A. E. Raftery (2007). "Strictly Proper Scoring Rules, Prediction, and Estimation". In: *Journal of the American Statistical Association* 102.477, pp. 359–378. DOI: 10.1198/016214506000001437.

Gu, A. and T. Dao (2024). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". In: *Conference on Language Modeling (COLM)*. DOI: 10.48550/arXiv.2312.00752.

Gu, A., K. Goel, and C. Ré (2022). "Efficiently Modeling Long Sequences with Structured State Spaces". In: *The Tenth International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.2111.00396.

Hamilton, J. D. (1994). *Time Series Analysis*. Princeton, NJ: Princeton University Press. ISBN: 9780691042893. DOI: 10.1515/9780691218632.

He, K. et al. (2015). "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.

Hendrycks, D. and K. Gimpel (2016). “Gaussian Error Linear Units (GELUs)”. In: *arXiv preprint arXiv:1606.08415*. DOI: 10.48550/arXiv.1606.08415.

Hochreiter, S. and J. Schmidhuber (1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

Hopfield, J. J. (1982). “Neural Networks and Physical Systems with Emergent Collective Computational Abilities”. In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558. DOI: 10.1073/pnas.79.8.2554.

Hyndman, R. J., R. A. Ahmed, et al. (2011). “Optimal Combination Forecasts for Hierarchical Time Series”. In: *Computational Statistics & Data Analysis* 55.9, pp. 2579–2589. DOI: 10.1016/j.csda.2011.03.006.

Hyndman, R. J. and A. B. Koehler (2006). “Another Look at Measures of Forecast Accuracy”. In: *International Journal of Forecasting* 22.4, pp. 679–688. DOI: 10.1016/j.ijforecast.2006.03.001.

Kaufman, S. et al. (2012). “Leakage in Data Mining: Formulation, Detection, and Avoidance”. In: *ACM Transactions on Knowledge Discovery from Data* 6.4, 15:1–15:21. DOI: 10.1145/2382577.2382579.

Kidger, P. et al. (2020). “Neural Controlled Differential Equations for Irregular Time Series”. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., pp. 6696–6707. DOI: 10.48550/arXiv.2005.08926.

Lim, B. et al. (2021). “Temporal Fusion Transformers for Interpretable Multi-Horizon Time Series Forecasting”. In: *International Journal of Forecasting* 37.4, pp. 1748–1764. DOI: 10.1016/j.ijforecast.2021.03.012.

Loshchilov, I. and F. Hutter (2017). “SGDR: Stochastic Gradient Descent with Warm Restarts”. In: *5th International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.1608.03983.

— (2019). “Decoupled Weight Decay Regularization”. In: *7th International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.1711.05101.

Micikevicius, P. et al. (2018). “Mixed Precision Training”. In: *6th International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.1710.03740.

Nie, Y. et al. (2023). “A Time Series is Worth 64 Words: Long-Term Forecasting with Transformers”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.2211.14730.

Oord, A. van den et al. (2016). “WaveNet: A Generative Model for Raw Audio”. In: *arXiv preprint arXiv:1609.03499*. DOI: 10.48550/arXiv.1609.03499.

Oreshkin, B. N. et al. (2020). “N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting”. In: *8th International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.1905.10437.

Pascanu, R., T. Mikolov, and Y. Bengio (2013). “On the Difficulty of Training Recurrent Neural Networks”. In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Vol. 28. Proceedings of Machine Learning Research. PMLR, pp. 1310–1318.

Ramsauer, H. et al. (2021). “Hopfield Networks Is All You Need”. In: *9th International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.2008.02217.

Rubanova, Y., R. T. Q. Chen, and D. K. Duvenaud (2019). “Latent ODEs for Irregularly-Sampled Time Series”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., pp. 5320–5330. DOI: 10.48550/arXiv.1907.03907.

Saidd, Nabeel Ahmad (2026a). “A Controlled Comparison of Deep Learning Architectures for Multi-Horizon Financial Forecasting: Evidence from 918 Experiments”. In: *arXiv preprint arXiv:2603.16886*. DOI: 10.48550/arXiv.2603.16886. arXiv: 2603.16886 [q-fin.ST].

— (2026b). “Do Technical Indicators Improve Deep Learning Forecasts? An Empirical Ablation Study Across Asset Classes”. In: *Preprints.org*. DOI: 10.20944/preprints202602.0692.v1.

— (2026c). “RNAD: A Regime-Normalized, Tail-Robust and Direction-Aware Loss for Deep Time-Series Forecasting”. In: DOI: 10.2139/ssrn.7177678.

Salinas, D. et al. (2020). “DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks”. In: *International Journal of Forecasting* 36.3, pp. 1181–1191. DOI: 10.1016/j.ijforecast.2019.07.001.

Schlag, I., K. Irie, and J. Schmidhuber (2021). “Linear Transformers Are Secretly Fast Weight Programmers”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. Proceedings of Machine Learning Research. PMLR, pp. 9355–9366.

Smith, J. T. H., A. Warrington, and S. W. Linderman (2023). “Simplified State Space Layers for Sequence Modeling”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.2208.04933.

Srivastava, N. et al. (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1, pp. 1929–1958. DOI: 10.5555/2627435.2670313.

Tashman, L. J. (2000). “Out-of-Sample Tests of Forecasting Accuracy: An Analysis and Review”. In: *International Journal of Forecasting* 16.4, pp. 437–450. DOI: 10.1016/S0169-2070(00)00065-0.

Tsay, R. S. (2010). *Analysis of Financial Time Series*. 3rd ed. Hoboken, NJ: John Wiley & Sons. ISBN: 9780470414354. DOI: 10.1002/9780470644560.

Vaswani, A. et al. (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., pp. 5998–6008. DOI: 10.48550/arXiv.1706.03762.

Voelker, A., I. Kajić, and C. Eliasmith (2019). “Legendre Memory Units: Continuous-Time Representation in Recurrent Neural Networks”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc.

Wang, Y. et al. (2024). “TimeXer: Empowering Transformers for Time Series Forecasting with Exogenous Variables”. In: *Advances in Neural Information Processing Systems*. Vol. 37. Curran Associates, Inc., pp. 469–498. DOI: 10.52202/079017-0015.

Williams, R. J. and J. Peng (1990). “An Efficient Gradient-Based Algorithm for On-Line Training of Recurrent Network Trajectories”. In: *Neural Computation* 2.4, pp. 490–501. DOI: 10.1162/neco.1990.2.4.490.

Wu, H., T. Hu, et al. (2023). “TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. OpenReview.net. DOI: 10.48550/arXiv.2210.02186.

Wu, H., J. Xu, et al. (2021). “Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting”. In: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., pp. 22419–22430. DOI: 10.48550/arXiv.2106.13008.

Zhou, H. et al. (2021). “Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12. AAAI Press, pp. 11106–11115. DOI: 10.1609/aaai.v35i12.17325.

Zhou, T. et al. (2022). “FEDformer: Frequency Enhanced Decomposed Transformer for Long-Term Series Forecasting”. In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. Vol. 162. Proceedings of Machine Learning Research. PMLR, pp. 27268–27286. DOI: 10.48550/arXiv.2201.12740.

A Technical and Reproducibility Appendix

This appendix collects material that supports reproduction and technical scrutiny but would interrupt the main argument: the event-transition derivation underlying Eqs. (2) and (4), tensor shapes, the ordered per-event algorithm, the configuration inventory across the three training scripts, and the per-seed metrics (Sections A.1 to A.5). All values are configuration values or values already reported in the main text, and no new experiments are introduced.

A.1 Event-transition derivation

Section 3 states the masked transition in terms of the scalar update indicator u_τ^k (Eq. (2)) and in the batched form actually implemented (Eq. (4)). The equivalent case-based form is closer to the code and makes explicit that masking selects between two already-computed branches rather than modifying either. Let $\bar{t}_\tau^k = \max\{t : t \le \tau,\ t \text{ is a completed level-}k \text{ timestamp}\}$ be the no-later-than selection rule and $u_\tau^k = \mathbf{1}\{\bar{t}_\tau^k = \tau\}$ the resulting indicator of Eq. (1). For a candidate transition Φ_k,

$$(r_{\tau+1}^k, M_{\tau+1}^k) = \begin{cases} \Phi_k(r_\tau^k, M_\tau^k, X_\tau^k), & u_\tau^k = 1, \\ (r_\tau^k, M_\tau^k), & u_\tau^k = 0, \end{cases} \tag{29}$$

which coincides with Eq. (2) because $u_\tau^k \in \{0, 1\}$, so the convex combination always selects exactly one branch. In the batched implementation the same selection is applied per sample: entry b of the mask vector m_τ^k (Eq. (3)) is the scalar indicator of stream b, and Eq. (4) broadcasts it over the state and memory dimensions. The base level has $u_\tau^1 \equiv 1$, since the anchor stream is defined by level-1 bar completions. This establishes preservation of persistent state, not computational skipping: candidate encodings are formed for all supplied windows regardless of the mask (Section 5).

The causal contract of Eq. (5) is event-level. The lower-level read is appended to its history before the higher-level evidence readout (Eq. (23)), so no future *event* is used, but a higher-level update can consume a lower-level representation produced at the same event. This is the reason for the qualified entry 9 of table 6, and it is the one place in the specification where "causal" must be read at event granularity rather than at the granularity of individual computation steps within an event.

A.2 Tensor shapes across the pipeline

Table 7 traces the tensor shape at each stage of Section 4 for the default configuration ($d_s = 72$, $d_m = 18$, window length $T = 32$, rank $d_r = 36$).

Table 7. Tensor shapes at each pipeline stage in the default configuration ($d_s = 72$, $d_m = 18$, $T = 32$, $d_r = 36$). B is the batch size and L the number of configured levels for the asset in question.

Stage	Shape
Raw window X^k	$B \times T \times 5$
Derived features d_t^k	$B \times T \times 6$
Encoder output e^k (Eq. (12))	$B \times 72$
Update mask m_τ^k (Eq. (3))	B
Memory input x_t (base / higher level)	$B \times 144$ / $B \times 216$
Memory M_t	$B \times 72 \times 18$
State r_t	$B \times 72$
Resonance stack R	$B \times L \times 72$
Evidence readout E_i (Eq. (23))	$B \times 72$
Head input (Eq. (24))	$B \times 150$
Forecast $\hat{y}_{t+1}^k$	$B \times 1$

At the base level, the memory input concatenates a 72-dimensional encoder output with a 72-dimensional prior resonance context. Higher levels additionally receive 72-dimensional bottom-up evidence, which accounts for the 144- versus 216-dimensional memory input of Section 4.2. The read $\bar{v} = Mk$ and residual $s = v - \bar{v}$ enter Eqs. (17) and (19) as described there. The shapes make explicit that the association store is a 72×18 matrix per sample, not a 72-dimensional vector state, which is the structural difference from a standard recurrent cell noted in Section 2.

A.3 Ordered per-event algorithm

The exact ordering of the per-event computation described in Section 5 is what makes the causal qualification of Section 3 (entry 9 of table 6) precise. It is restated here as an ordered procedure:

1. select the latest completed window and update indicator for each level (Section 3);
2. encode all supplied windows and construct anchor features (Section 4);
3. apply prior-state resonance (Eq. (22));
4. update the base level, which is always active;
5. append each current lower-level read and compute the adjacent higher-level evidence (Eq. (23));
6. compute candidate higher-level transitions and apply their masks (Eq. (4));
7. apply posterior resonance to the resulting states (Eq. (22)); and
8. emit one BPS forecast per configured level (Eq. (24)).

Steps 3 and 7 are the two resonance applications of Section 4, and step 5 is the ordered evidence computation responsible for the qualified entry 9. Price reconstruction occurs only after the BPS forecast is emitted in step 8, consistent with the network being optimized purely in BPS space.

A.4 Configuration inventory across the training scripts

The production trainer, the reported configuration table, and the ablation trainer do not share identical defaults (Sections 5 and 6.2), and these artifact-level configurations must not be silently merged. Table 8 consolidates the values referenced individually in the main text.

Table 8. Key training settings of the production trainer, the reported main-results recipe, and the ablation trainer. The configurations differ only in the seed lists and in the ablation epoch budget; shared values are shown explicitly rather than inferred.

Setting	Production	Main results	Ablations
Batch size	256	256	256
Max epochs	50	50	5
Evaluation streams	8	8	8
Optimizer	AdamW	AdamW	AdamW
Learning rate	10^{-3}	10^{-3}	10^{-3}
Weight decay	10^{-5}	10^{-5}	10^{-5}
Gradient clipping	1.0	1.0	1.0
Patience	15	15	15
Schedule	Cosine	Cosine	Cosine
Seeds	42	42, 151, 359	42
Precision	CUDA AMP	CUDA AMP	CUDA AMP

One discrepancy should be resolved from the available configuration and checkpoint metadata: the reported recipe lists eight-stream validation and test, whereas the standalone evaluator uses one stream and reconstructs missing checkpoint configuration fields from defaults and tensor shapes. Because the production trainer writes seed-specific checkpoints and the dataset configuration is available in the repository, the reported three-seed aggregates can be regenerated from the repository together with a compatible vendor feed. Per-seed aggregate metrics and seed-specific checkpoints are available, and per-event outputs can be generated from those checkpoints. No target-space metric is emitted by the evaluators, and no persistence baseline, confidence interval, or paired test is provided (Section 7). The ablation variants A1–A3 are interface-compatible interventions, not capacity-controlled counterfactuals (Section 8).

A.5 Per-seed metrics

The per-seed aggregate metrics reported here are for seeds 42, 151, and 359. They are aggregate metrics, not per-event predictions. Table 9 lists HARN's per-seed metrics for each asset and timeframe, and Table 10 lists those of PatchTST and TimeXer. The baseline per-seed CSV does not contain a timeframe column, so the timeframe labels in Table 10 are inferred from Tables 2 and 4. The mean ± standard deviation entries in Table 4 are computed from these per-seed values, and the reported standard deviations follow the population convention, in which the sum of squared deviations is divided by the number of seeds. Recomputing the means and standard deviations from Tables 9 and 10 reproduces the corresponding entries in Table 4, and no discrepancies were found. Per-seed training trajectories and

per-event predictions are not included as precomputed files and can be generated from the checkpoints and preprocessing pipeline.

Table 9. HARN per-seed aggregate test metrics for seeds 42, 151, and 359, grouped by asset and timeframe. The means and standard deviations in Table 4 are computed from these values.

Asset	TF	Seed	MAE	RMSE	sMAPE	MASE
AAPL	H1	42	1.0588	1.8011	0.4260	4.8159
		151	1.0603	1.8002	0.4265	4.8230
		359	1.0581	1.7994	0.4258	4.8130
	M15	42	0.5070	0.9330	0.2047	1.3257
		151	0.5080	0.9334	0.2051	1.3283
		359	0.5081	0.9340	0.2051	1.3287
EURUSD	H1	42	0.0006	0.0009	0.0530	3.4689
		151	0.0006	0.0009	0.0531	3.4742
		359	0.0006	0.0009	0.0527	3.4460
	H4	42	0.0012	0.0017	0.1055	13.5457
		151	0.0012	0.0017	0.1053	13.5221
		359	0.0012	0.0017	0.1056	13.5625
	M15	42	0.0003	0.0005	0.0262	0.8612
		151	0.0003	0.0005	0.0266	0.8745
		359	0.0003	0.0005	0.0262	0.8614
USDCHF	H1	42	0.0005	0.0008	0.0667	3.2374
		151	0.0005	0.0008	0.0665	3.2244
		359	0.0005	0.0008	0.0667	3.2364
	M15	42	0.0003	0.0004	0.0333	0.8002
		151	0.0003	0.0004	0.0333	0.7997
		359	0.0003	0.0004	0.0333	0.8005
	M30	42	0.0004	0.0005	0.0473	1.6103
		151	0.0004	0.0005	0.0471	1.6049
		359	0.0004	0.0005	0.0472	1.6077
XAUUSD	M15	42	4.5324	6.8201	0.1085	3.4851

Asset	TF	Seed	MAE	RMSE	sMAPE	MASE
		151	4.5239	6.8158	0.1083	3.4786
		359	4.5301	6.8268	0.1084	3.4833
		42	2.5500	3.8445	0.0610	1.1351
	M5	151	2.5535	3.8459	0.0611	1.1366
		359	2.5529	3.8424	0.0611	1.1363

Table 10. Per-seed aggregate test metrics for PatchTST and TimeXer, grouped by asset, model, and timeframe. Timeframe labels are inferred from Tables 2 and 4 because the source CSV has no timeframe column.

Asset	Model	TF	Seed	MAE	RMSE	sMAPE	MASE
			42	1.3590	1.9500	0.5278	3.5536
	PatchTST	M15	151	2.0955	2.8652	0.7919	5.4796
			359	2.7687	3.2913	1.0643	7.2400
AAPL			42	3.8550	4.6007	1.4807	10.0806
	TimeXer	M15	151	1.8682	2.5298	0.7647	4.8852
			359	2.9224	3.7099	1.1004	7.6420
			42	0.0007	0.0008	0.0564	1.8593
	PatchTST	M15	151	0.0011	0.0014	0.0971	3.2066
			359	0.0005	0.0007	0.0416	1.3668
EURUSD			42	0.0112	0.0137	0.9635	31.7270
	TimeXer	M15	151	0.0104	0.0124	0.8911	29.3482
			359	0.0109	0.0130	0.9355	30.8055
			42	0.0008	0.0010	0.0992	2.3719
	PatchTST	M15	151	0.0006	0.0008	0.0722	1.7386
			359	0.0006	0.0008	0.0714	1.7129
USDCHF			42	0.0145	0.0160	1.8288	44.1695
	TimeXer	M15	151	0.0134	0.0148	1.6934	40.8628
			359	0.0145	0.0162	1.8257	44.0738
			42	8.6349	11.6743	0.2056	3.8436
	PatchTST	M5	151	11.0156	13.9459	0.2640	4.9034
XAUUSD							

Asset	Model	TF	Seed	MAE	RMSE	sMAPE	MASE
			359	9.0399	12.4158	0.2154	4.0239
	TimeXer	M5	42	16.2410	21.2480	0.3801	7.2293
			151	16.8017	21.0780	0.3956	7.4789
			359	12.8497	14.9588	0.3081	5.7197